A PRACTICAL COURSE

Async PostgreSQL in Rust

tokio-postgres from the first query to a production-shaped axum service — for developers who know Rust, a little Tokio and some SQL.

7 chapters · every runnable example was compiled and run against PostgreSQL as written

python3 tools/book.py book

Contents

0 Introduction and setup . . . **1**

Contents . . . 1

Who this is for, and what it assumes . . . 2

Where tokio-postgres fits . . . 2

How each chapter is built . . . 3

stockroom in one page . . . 4

Setting up . . . 5

A ten-line taste . . . 7

Exercises . . . 8

1 Connecting . . . **9**

1.1 Two halves: `Client` and `Connection` . . . 9

1.2 Spawning the connection, and forgetting to . . . 10

1.3 Connection strings . . . 11

1.4 The `Config` builder . . . 13

1.5 What the server told you, and how a connection ends . . . 14

1.6 What failure looks like . . . 15

1.7 The playground's `connect` . . . 18

In stockroom . . . 18

Summary . . . 19

Exercises . . . 20

2 Running SQL: execute, query and parameters . . . **22**

2.1 `batch_execute`: scripts . . . 22

2.2 `execute`: changes, counted . . . 23

2.3 Parameters . . . 24

2.4 `query`, `query_one` and `query_opt`: reading . . . 26

2.5 Scalars: `query_scalar`, `query_one_scalar`, `query_opt_scalar` . . . 28

2.6 `RETURNING`: changing and reading in one statement . . . 28

2.7 The simple query protocol . . . 29

In stockroom . . . 31

Summary . . . 32

Exercises . . . 33

3 Rows and the type mapping . . . **35**

3.1 Reading a `Row` . . . 35

3.2 The type table 36
3.3 NULL 38
3.4 Borrowing from a row 39
3.5 Rows into structs 40
3.6 Dates and times 41
3.7 JSON 42
3.8 Arrays 43
3.9 Exact numbers, UUIDs 45
In stockroom 47
Summary 47
Exercises 48

4 Prepared statements and the extended protocol 50
4.1 What `query` sends 50
4.2 `prepare`: one parse, many executions 51
4.3 When the server cannot infer a type 52
4.4 `query_typed`: one round trip, no statement 53
4.5 Counting round trips 54
4.6 Caching statements, and when the cache goes stale 55
In stockroom 58
Summary 59
Exercises 59

5 Custom types: ToSql and FromSql 62
5.1 The two traits 62
5.2 A newtype by hand 63
5.3 Deriving: `transparent` newtypes 64
5.4 Enums 65
5.5 Domains 67
5.6 Composite types 69
5.7 Behind the scenes: `Type`, `Kind` and the type cache 70
In stockroom 72
Summary 73
Exercises 73

6 Errors 75
6.1 One type, many causes 75
6.2 `DbError`: the server's report 77
6.3 `SqlState`: the codes worth matching 79
6.4 `is_closed`, and the error you cannot classify 81
6.5 An error type for an application 81
6.6 An error inside a transaction 83

6.7 Notices are not errors . . . 84
In stockroom . . . 86
Summary . . . 87
Exercises . . . 87

CHAPTER 0

Introduction and setup

This book teaches `tokio-postgres`, the asynchronous PostgreSQL driver that most of Rust's database ecosystem stands on. It takes you from the first query to a production-shaped HTTP service built with axum, and at every step it shows what the driver sends over the wire, what the server does with it, and where the sharp edges are.

Every runnable example was compiled and run against PostgreSQL 18 when the book was written, and the service the book builds, **stockroom**, is in this repository with its tests. You learn the driver and a real codebase at the same time.

Contents

Chapter	You will learn
0 · Introduction and setup (this page)	Who the book is for, where tokio-postgres fits, the setup, and stockroom in one page
1 · Connecting	`Client` and `Connection`, spawning the connection, connection strings and `Config`, what failure looks like
2 · Running SQL	`execute`, `query` and friends, parameters instead of string formatting, the simple query protocol
3 · Rows and types	Reading rows, the Rust–PostgreSQL type table, NULL, JSON, arrays, borrowing from a row
4 · Prepared statements	What goes over the wire, `Statement`, type inference, `query_typed`, round trips
5 · Custom types	`ToSql` and `FromSql` by hand and by derive: newtypes, enums, domains, composites
6 · Errors	`Error`, `DbError` and `SqlState`, and an error type for an application
7 · Pipelining	Many queries on one connection at once, and when that beats a pool
8 · Transactions	Isolation levels, savepoints, retries, locks, and selling the last chair exactly once
9 · Streaming	Rows as a stream, backpressure, portals, keyset pagination
10 · COPY	Bulk data in and out, text and binary, and how much faster it is
11 · LISTEN/NOTIFY	Driving the connection yourself, notifications, notices, reconnecting
12 · Production connections	Timeouts, cancellation, dead connections, failover and TLS

Chapter	You will learn
13 · Pooling	Why one client is not enough, a pool from scratch, deadpool-postgres
14 · The data layer	Migrations, transactional DDL, a repository layer
15 · axum I	A JSON API: extractors, errors as responses, ETags, pagination
16 · axum II	Streaming bodies, COPY over HTTP, server-sent events, an outbox worker, shutdown
17 · Testing	A database per test, HTTP tests, concurrency tests
18 · Capstone	stockroom end to end, and a map from every feature to its code
A · Appendix	Cheat sheets, a sqlx and diesel phrasebook, a glossary

The whole book is also available as a single PDF, tokio-postgres-book.pdf, with a linked table of contents. To print one chapter, render it yourself from the repository's root: `python3 tools/book.py pdf 3` writes `target/book/03-rows-and-types.pdf` (it needs pandoc and Typst; the README's *PDFs* section has the details).

Chapters 1–6 are the driver's foundations. Chapters 7–12 are what makes it an *async* driver, and where most production problems live. Chapters 13–18 build the service. Read them in order the first time; afterwards the appendix is the fastest way back in.

Who this is for, and what it assumes

You know three things well enough to be dangerous:

- **Rust, at an average level.** Ownership and borrowing, structs and enums with `match`, traits and generics, `Result` and `?`, closures, iterators. You have probably not had to think hard about `Pin`, higher-ranked lifetimes or why a future is not `Send`. The book explains each of those where it first matters.
- **Tokio, at a basic level.** You have written `#[tokio::main]`, `.await`ed things, called `tokio::spawn`, and seen a channel. `select!`, cancellation and streams are introduced as they are needed.
- **SQL, at a basic level.** `SELECT`, `INSERT`, `UPDATE`, `DELETE`, joins, `CREATE TABLE`, and the idea of a transaction. Isolation levels, row locks, `COPY`, `LISTEN/NOTIFY`, domains and composite types are explained when they come up.

You do not need to know anything about how a database driver works. By the end you will know what yours does on every call.

Where tokio-postgres fits

Rust has several ways to talk to PostgreSQL, and they are layered more than they compete:

Library	What it is	Built on
tokio-postgres	An asynchronous driver: connections, queries, transactions, COPY, notifications, type conversions. No query builder, no ORM, no compile-time checking.	Tokio and the PostgreSQL wire protocol
`postgres`	The same driver with a blocking API, for programs without an async runtime.	tokio-postgres
deadpool-postgres, bb8-postgres	Connection pools.	tokio-postgres
refinery	Schema migrations.	tokio-postgres (among others)
clorinde (formerly cornucopia)	Generates typed Rust functions from `.sql` files.	tokio-postgres
diesel-async (PostgreSQL backend)	Diesel's query builder, asynchronously.	tokio-postgres
sqlx	An independent async driver with compile-time-checked queries.	its own protocol implementation
SeaORM	An ORM.	sqlx

Learning tokio-postgres pays off twice. It is a good driver to use directly: small, close to the protocol, and fast, with features higher layers hide, such as running many queries on one connection at once (Chapter 7). And whatever you use on top of it, it is what actually runs when something goes wrong in production, so knowing it is knowing what your stack does.

How each chapter is built

1. **The concept**, explained from scratch, with small runnable programs.
2. **Side by side** sections. Each one states a single task and solves it three ways: at the `psql` prompt, with sqlx, and with tokio-postgres. A table of what is the **same** and what is **different** follows. `psql` ties the Rust code to SQL you already know and shows what the server is asked to do; sqlx shows which choices are specific to tokio-postgres. The sqlx snippets are illustrations and are not compiled; the tokio-postgres ones are complete and were run. In Part III, where psql has nothing to compare with, the sections drop it.
3. **In stockroom**: where the concept lives in the service this book builds, with a trimmed excerpt and a link to the file.
4. **Exercises**, each with a hint, and solutions for the ones whose answers are not obvious.

Four kinds of boxes appear where they are earned:

> **Protocol lens:** what travels over the socket. Most of tokio-postgres's API makes sense the moment you know what it sends.

> **SQL lens:** what the server does: isolation, locks, how notifications are delivered.

> **Rust refresher:** a language feature you may be hazy on, at the point where it matters.

> **Pitfall:** a mistake that compiles, passes a quick test, and fails in production.

Code blocks come in two kinds:

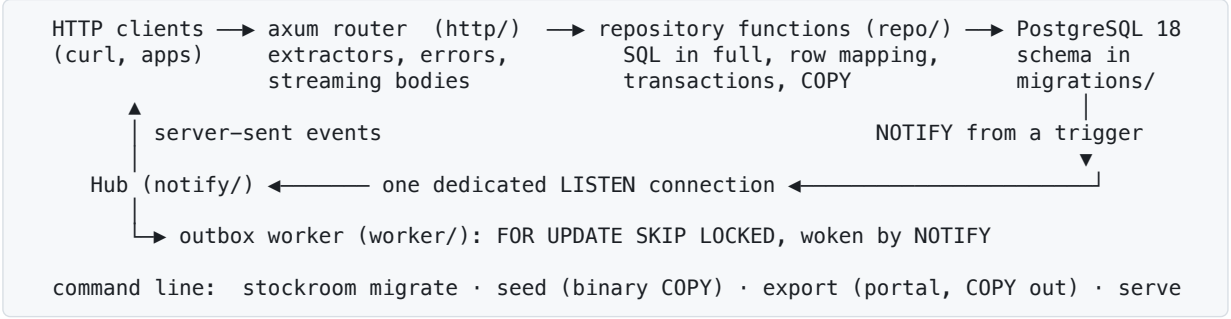
- Blocks marked as plain `rust` are **complete programs**. Each one compiles and runs as written in the playground described below, and each was run against a fresh database when the book was written. Their `assert!`s are part of the explanation: they state what the code produces.
- Blocks taken from stockroom or the example programs are **excerpts**. They compile in their own crate, not on their own, and their first line says where they come from.

(For maintainers: in the Markdown source, runnable examples are fenced with backticks and everything else with tildes, `~~~rust`, so that `tools/book.py` can extract and run every runnable block, and check every excerpt against the file it quotes. An HTML comment before a block, such as `<!-- check: needs=lagproxy -->`, tells the tool how to run it.)

---

stockroom in one page

stockroom is a click-and-collect service for a small retail chain: a catalog, stock per store, and orders that customers place online and pick up in a store. It is small enough to read in an afternoon and real enough to have the problems real services have: two customers wanting the last chair, staff screens that must update live, price lists arriving as CSV files, emails that must go out exactly when an order is ready.



Part of stockroom	What it is good for teaching	Chapters
domain/	Mapping a PostgreSQL enum, a composite type and two domains to Rust types, by hand and by derive	3, 5
error.rs	Turning SQLSTATE codes and constraint names into HTTP responses	6, 15
repo/	SQL written out in full, pipelined queries, transactions, COPY, savepoints	2–10, 14
db/	Connecting, pooling, migrations, retries, cancelling abandoned queries, TLS	1, 8, 12–14
notify/	A connection driven by hand, reconnecting, fan-out to many clients	11, 16
http/	axum on top of all of it	15, 16
tests/	A database per test, and proving that no order oversells	17

Five small programs in apps/ each do one thing a chapter talks about: pgq is a tiny psql, lagproxy puts a slow network between you and the server, pgload bulk-loads rows five ways and times them, pglisten tails notifications, and minipool is a connection pool in about a hundred lines.

Setting up

PostgreSQL

The book targets **PostgreSQL 18**. Most examples run on 14 or newer; the text says so whenever a feature needs a newer server (for example, "`MERGE` (PG 15+)"). Any of these will do:

```
# The EDB installer (macOS, Windows, Linux): https://www.postgresql.org/download/
# Homebrew (macOS):
brew install postgresql@18 && brew services start postgresql@18
# Docker:
docker run --name pg18 -e POSTGRES_PASSWORD=secret -p 5432:5432 -d postgres:18
```

You also want `psql`, PostgreSQL's command-line client. It comes with all three; with Docker, `docker exec -it pg18 psql -U postgres` works without installing anything.

Create a database for the book's examples, and tell the examples where it is with an environment variable:

```
psql "postgres://postgres:secret@localhost:5432/postgres" -c 'CREATE DATABASE tpbook'
export DATABASE_URL=postgres://postgres:secret@localhost:5432/tpbook
```

Use your own user, password and port. The examples only create temporary tables, which disappear when their connection closes, so they leave nothing behind and can run in any order.

The toolchain

The book uses Rust's 2024 edition, so you need Rust 1.85 or newer (it was written with 1.99):

```
rustup update
rustc --version
```

The playground

Make a scratch crate for running the book's examples. It must be called `playground`, because the examples use a helper from it:

```
cargo new --lib playground
cd playground
```

Replace its `Cargo.toml` with this. These are the crates the book's runnable examples use. The versions are minimums: any newer compatible release works, and `cargo update` fetches the latest one.

```
[package]
name = "playground"
version = "0.1.0"
edition = "2024"
publish = false

[dependencies]
anyhow = "1.0.104"
async-stream = "0.3.6"
axum = "0.8.9"
bytes = "1.12.1"
deadpool-postgres = "0.14.2"
futures-util = "0.3.34"
http-body-util = "0.1.5"
jiff = { version = "0.2.39", features = ["serde"] }
postgres-types = { version = "0.2.14", features = ["derive"] }
rust_decimal = { version = "1.43.0", features = ["db-tokio-postgres"] }
serde = { version = "1.0.229", features = ["derive"] }
serde_json = "1.0.154"
thiserror = "2.0.21"
tokio = { version = "1.54.1", features = ["full"] }
tokio-postgres = { version = "0.7.18", features = ["with-jiff-0_2", "with-serde_json-1",
"with-uuid-1", "array-impls"] }
tower = { version = "0.5.3", features = ["util"] }
uuid = { version = "1.29.0", features = ["v7", "serde"] }
```

The `features` on `tokio-postgres` switch on conversions for other crates' types: `jiff`'s timestamps, `serde_json`'s values, `uuid`'s UUIDs, and Rust arrays. Chapter 3 explains them.

Then replace `src/lib.rs` with the helper every example uses to connect. Chapter 1 explains every line of it:

```
use tokio_postgres::{Client, Error, NoTls};

/// Connects to the database named by the `DATABASE_URL` environment variable and spawns the
/// connection task, returning the client.
///
/// # Panics
///
/// If `DATABASE_URL` is not set: examples are programs, not libraries, and a clear panic is
/// the most useful thing they can do about a missing setting.
pub async fn connect() -> Result<Client, Error> {
    let url = std::env::var("DATABASE_URL")
        .expect("set DATABASE_URL, e.g. postgres://postgres:secret@localhost:5432/tpbook");
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;
    tokio::spawn(async move {
        if let Err(e) = connection.await {
            eprintln!("connection error: {e}");
        }
    });
    Ok(client)
}
```

Put each example in its own file under `examples/`, for instance `examples/ch00_taste.rs`, and run it with

```
cargo run --example ch00_taste
```

The commands you will use in this repository

```
cargo run -p pgq -- -c "SELECT version()"     # the tiny psql from apps/pgq
cargo run -p stockroom -- --help              # the service's command line
cargo test -p stockroom                       # its tests: they create and drop their own
databases
python3 tools/book.py check                   # run every example in this book
```

The repository's own `.env.example` lists the environment variables they read. Copy it to `.env` and fill in your credentials.

A ten-line taste

Connect, send one query with a parameter, and read two typed values out of the row:

```
use playground::connect;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = connect().await?;

    // `$1` is a parameter: its value travels separately from the SQL text.
    let row = client
        .query_one(
            "SELECT $1::text || ', ' || current_user AS greeting, 6 * 7 AS answer",
            &[&"hello"],
        )
        .await?;

    let greeting: &str = row.get("greeting");
    let answer: i32 = row.get("answer");
    println!("{greeting}; the answer is {answer}");
    assert!(greeting.starts_with("hello, "));
    assert_eq!(answer, 42);
    Ok(())
}
```

```
hello, postgres; the answer is 42
```

Four things in these lines are worth noticing now, and each gets a chapter later:

- `connect().await?` gives you a `Client`, but something else must keep the connection alive in the background: the helper spawns it. Forgetting that is the first mistake everyone makes (Chapter 1).
- The value `"hello"` is not pasted into the SQL. It is sent as a **parameter**, `$1`, in a separate message, which is why SQL injection cannot happen here (Chapter 2).
- `row.get("answer")` turns PostgreSQL's `integer` into Rust's `i32` because you asked for an `i32`. Ask for an `i64` and it panics: the conversion is checked, at run time (Chapter 3).
- `query_one` sent more than one message and waited for more than one answer. Knowing how many is the key to making a database-backed service fast (Chapter 4).

Exercises

1. **Set up the playground** as described above, save the ten-line taste as `examples/ch00_taste.rs`, and run it. *Hint:* if it panics with "set DATABASE_URL", the environment variable is not set in the shell you ran `cargo` from.
2. **Break it on purpose.** Change `let answer: i32` to `let answer: i64` and run it again. Read the panic message: what does it say about the types on each side? *Hint:* the SQL expression `6 * 7` has the type `integer`, which is 32 bits wide.
3. **Use the tiny psql.** From this repository's root, run `cargo run -p pgq -- -c "SELECT name, setting FROM pg_settings WHERE name LIKE 'max_conn%'"` and then the same with `--describe`. What does `--describe` tell you that the plain run does not?
4. **Run the service.** From the repository's root: `cargo run -p stockroom -- migrate`, then `cargo run -p stockroom -- seed`, then `cargo run -p stockroom -- serve`, and open stockroom.http in an editor with an HTTP client (VS Code's REST Client, or curl by hand). Place an order and watch it in another terminal with `curl -N 'http://127.0.0.1:8080/orders/events?store=store-01'`. *Hint:* `seed` creates the stores `store-01` to `store-03`.

Solutions

2. The panic reads only "error retrieving column answer: error deserializing column 1". The reason is one level down, in the error's `source()`, which a panic message does not print. With `row.try_get::<_, i64>("answer")` instead of `get`, you can print the error with `{:?}`:

```
Error { kind: FromSql(1), cause: Some(WrongType { postgres: Int4, rust: "i64" }) }
```

tokio-postgres knows the column's type from the row description the server sent, and every Rust type says which PostgreSQL types it accepts. `i64` takes only `int8`. There is no silent widening: you change the Rust type, or cast in SQL (`(6 * 7)::int8`). Chapter 6 shows how to get at the cause of any error.

3. `--describe` does not run the query. It asks the server to prepare it and reports the type of every result column (both `text` here) and, for columns that come from a table or view, which one: `oid 12104, column 1` is the first column of the view `pg_settings`, whose OID (object id) you can look up with `SELECT 12104::regclass`. The plain run uses the simple query protocol, in which every value arrives as text and types are not part of the result (Chapter 2).

CHAPTER 1

Connecting

Every other chapter starts with a connection, so this one looks at what a connection *is* in tokio-postgres, and why it comes in two halves. That split is the most important design decision in the crate. It is why one connection can be shared by many tasks, why queries can be pipelined (Chapter 7), and why notifications need special handling (Chapter 11). It is also the reason for the first bug everyone writes: a program that connects successfully and then hangs forever on its first query.

The rest of the chapter is practical: connection strings and the `Config` builder, what the server tells you when you connect, how a connection ends, and what each kind of failure looks like, so you recognize them when they happen.

1.1 Two halves: `Client` and `Connection`

`tokio_postgres::connect` returns a pair:

```
use tokio_postgres::NoTls;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");

    // Two values: the client you call methods on, and the connection that does the IO.
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;

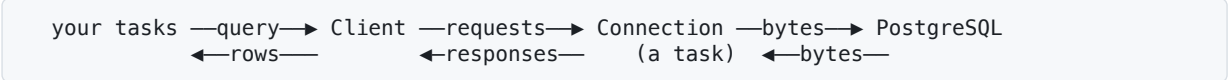
    // The connection is a future. Until something polls it, nothing is sent or received:
    // hand it to the runtime as a task of its own.
    let task = tokio::spawn(connection);

    let version: String = client.query_one("SHOW server_version", &[]).await?.get(0);
    println!("connected to PostgreSQL {version}");
    assert!(!version.is_empty());

    // Dropping the client tells the connection there is nothing more to do: it says goodbye
    // to the server and its future completes, with `Ok(())`.
    drop(client);
    task.await.expect("the task did not panic")?;
    Ok(())
}
```

`NoTls` says "no encryption"; Chapter 12 replaces it with a real TLS connector. The rest is the shape of every program in this book.

The two halves divide the work like this:



- The **`Connection`** owns the socket. It is a `Future` that, as long as it is polled, writes queued requests to the server, reads whatever the server sends, and routes each response to whoever is waiting for it. It finishes when the client is gone, or with an error when the connection breaks.
- The **`Client`** owns no socket at all. It holds the sending end of a channel to the connection. A method such as `query_one` turns your call into protocol messages, puts them on the channel, and waits for its own response to come back.

Two consequences matter right away. First, the `Client`'s methods take `&self`, not `&mut self`, so one client can be shared by many tasks at once (wrapped in an `Arc`); their requests queue up on the same connection. Second, nothing happens unless the connection is polled, which brings us to the first bug.

> **Rust refresher:** a `Future` is a value that describes work; it does nothing until it is polled, and `.await` is one way to poll it. `tokio::spawn(fut)` is another: it hands the future to the runtime, which polls it in the background, on any of its threads. That is why a spawned future must be `Send` (movable between threads) and `'static` (owning its data). `Connection` is both, by design.

1.2 Spawning the connection, and forgetting to

If you keep the connection but never poll it, your queries wait forever:

```
use std::time::Duration;
use tokio_postgres::NoTls;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;

    // Nobody polls `connection`. The request is queued for it, and nothing happens.
    let waited = tokio::time::timeout(
        Duration::from_millis(200),
        client.query_one("SELECT 1", &[]),
    )
    .await;
    assert!(
        waited.is_err(),
        "the query timed out: it never left the process"
    );

    // Start the connection, and the next query goes through. (So does the abandoned
    // `SELECT 1`: it was still queued, and its answer is thrown away.)
    tokio::spawn(connection);
    let one: i32 = client.query_one("SELECT 1", &[]).await?.get(0);
    assert_eq!(one, 1);
    Ok(())
}
```

Note what `connect` itself did: it opened the socket, logged in and waited for the server to be ready, all *before* returning. `connect().await` succeeding means the server accepted you. The connection's background work only begins once you spawn it.

Spawning is not the only way to poll a connection; it is just the convenient one. A short program can poll the connection and its query together, in the same task, with `select!`:

```rust
use tokio_postgres::NoTls;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;

    // Poll both: the query finishes first, because the connection only finishes when the
    // client is gone (or the server goes away).
    let answer: i32 = tokio::select! {
        row = client.query_one("SELECT 40 + 2", &[]) => row?.get(0),
        ended = connection => panic!("the connection ended first: {ended:?}"),
    };
    assert_eq!(answer, 42);
    Ok(())
}
```

That works, but it is fragile: the moment two queries are in flight, or one runs after the `select!`, the connection is no longer being polled. In real programs, spawn it, and do something with its result:

```rust
let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;
tokio::spawn(async move {
    if let Err(e) = connection.await {
        eprintln!("connection error: {e}");
    }
});
```

The error matters. When the server restarts or the network drops, this is where you hear about it first; every query in flight fails with "connection closed" a moment later.

> **Pitfall:** `tokio::spawn(connection);` alone compiles, runs, and throws the connection's error away (the `JoinHandle` is dropped, and with it the `Result`). In a service, log it.

> **Protocol lens:** what `connect` does before it returns. The client sends a *StartupMessage* with the user, the database and any settings. The server asks for a password (for a `scram-sha-256` server, a SCRAM exchange of two more round trips), then sends a series of *ParameterStatus* messages (the server's version, encoding, time zone and so on), one *BackendKeyData* message (a process id and a secret, kept for cancelling queries, Chapter 12), and finally *ReadyForQuery*. That is three to four round trips before your first query; on a slow network you feel it, which is one reason connections are pooled (Chapter 13).

1.3 Connection strings

`tokio_postgres::connect` takes a string in either of the two forms libpq, PostgreSQL's C library, understands:

- **Key/value pairs**, separated by spaces: `host=localhost port=5432 user=shop dbname=stock`. A value with spaces, or an empty one, goes in single quotes: `password='two words'`.

- **A URL**: `postgres://user:password@host:port/database?key=value&key=value`. Every key/value setting can go in the query string. Characters with a meaning in URLs must be percent-encoded: a password `s3cr@t` is written `s3cr%40t`.

Both parse into a `Config`, and you can parse one yourself to look inside:

```
use std::time::Duration;
use tokio_postgres::Config;
use tokio_postgres::config::Host;

fn main() -> Result<(), tokio_postgres::Error> {
    let from_url: Config =
        "postgres://shop:s3cr%40t@db.example.com:5433/stock?application_name=reports&
connect_timeout=5"
            .parse()?;
    let from_pairs: Config = "host=db.example.com port=5433 user=shop password='s3cr@t' \
                              dbname=stock application_name=reports connect_timeout=5"
        .parse()?;

    for config in [&from_url, &from_pairs] {
        assert_eq!(config.get_user(), Some("shop"));
        assert_eq!(config.get_password(), Some(&b"s3cr@t"[..])); // bytes, decoded
        assert_eq!(config.get_dbname(), Some("stock"));
        assert_eq!(config.get_hosts(), [Host::Tcp("db.example.com".into())]);
        assert_eq!(config.get_ports(), [5433]);
        assert_eq!(config.get_application_name(), Some("reports"));
        assert_eq!(config.get_connect_timeout(), Some(&Duration::from_secs(5)));
    }

    // Printing a Config never shows the password.
    let printed = format!("{from_url:?}");
    assert!(printed.contains("password: Some(_)") && !printed.contains("s3cr"));
    Ok(())
}
```

The settings you will meet most often:

Key	Meaning	Default
`host`	Host name or IP address; a value starting with `/` is a Unix-socket directory	required
`port`	TCP port (or the socket file's suffix)	5432
`user`, `password`	Who you log in as	none (`user` is required)
`dbname`	The database	the user's name
`application_name`	How the session shows up in `pg_stat_activity` and the server log	none
`connect_timeout`	Seconds to wait for each connection attempt	wait as long as the OS does
`options`	Settings for the session, as server command-line options: `-c statement_timeout=5000`	none
`sslmode`	`disable`, `prefer` or `require` (Chapter 12)	`prefer`

Several hosts can be listed, separated by commas (`host=primary,replica`), and they are tried in order. Chapter 12 uses that for failover.

> **Pitfall:** tokio-postgres reads **no environment variables** and no files. libpq-based tools such as psql also look at `PGHOST`, `PGPASSWORD`, `PGSERVICE`, `~/.pgpass` and more; a connection string that works in psql because `PGPASSWORD` is set will fail with "password missing" from Rust. Everything must be in the string or the `Config`. The convention for passing that string to a program is the `DATABASE_URL` environment variable, which your program reads itself, as the playground helper does.

1.4 The `Config` builder

When a program needs settings of its own on top of what the operator configured, parse the string and add to it. Every setter takes `&mut self` and returns `&mut Config`, so calls chain:

```
use std::time::Duration;
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    // Start from what the operator gave us...
    let mut config: Config = std::env::var("DATABASE_URL")
        .expect("DATABASE_URL is set")
        .parse()?;
    // ...and add what this program needs.
    config
        .application_name("chapter-1")
        .connect_timeout(Duration::from_secs(3))
        // Session settings, applied by the server before the first query.
        .options("-c statement_timeout=1500 -c TimeZone=UTC");

    let (client, connection) = config.connect(NoTls).await?;
    tokio::spawn(connection);

    let row = client
        .query_one(
            "SELECT current_setting('application_name'),
                    current_setting('statement_timeout'),
                    current_setting('TimeZone')",
            &[],
        )
        .await?;
    assert_eq!(row.get::<_, &str>(0), "chapter-1");
    assert_eq!(row.get::<_, &str>(1), "1500ms");
    assert_eq!(row.get::<_, &str>(2), "UTC");
    Ok(())
}
```

`options` deserves a moment. It is passed to the server at startup, so its settings are in force from the first statement of the session, without a round trip of their own, and a pool (Chapter 13) applies them to every connection it opens. stockroom uses it to give every session a `statement_timeout`, a `lock_timeout` and an `idle_in_transaction_session_timeout` (see *In stockroom* below).

Some settings are lists, and their setters *add* to them. That catches people who want to "point the configuration somewhere else":

```
use tokio_postgres::Config;
use tokio_postgres::config::Host;

fn main() -> Result<(), tokio_postgres::Error> {
    let mut config: Config = "postgres://app@db.internal:5432/shop".parse()?;
```

```
    // Meant: "connect to the local copy instead". Did: "try db.internal, then localhost".
    config.host("localhost").port(5433);
    assert_eq!(
        config.get_hosts(),
        [
            Host::Tcp("db.internal".into()),
            Host::Tcp("localhost".into())
        ]
    );
    assert_eq!(config.get_ports(), [5432, 5433]);

    // A Unix-domain socket is a host too: the directory that holds the socket file.
    let local: Config = "host=/tmp port=5432 user=app".parse()?;
    assert!(matches!(local.get_hosts(), [Host::Unix(dir)] if dir.as_os_str() == "/tmp"));
    Ok(())
}
```

Pitfall: `Config::host` and `Config::port` append; there is no setter that replaces the list. The first host still wins, so the mistake is silent: the program connects, just not where you think. To redirect a connection, build a fresh `Config` from the parts you need, or open the socket yourself and use `Config::connect_raw` (Chapter 12). The example programs in this repository do both, and both started with exactly this bug.

1.5 What the server told you, and how a connection ends

While logging you in, the server sent a list of its settings, and the `Connection` kept them. `Connection::parameter` reads them, and it is the only way to: once the connection has been moved into a task, you can no longer reach it, so read what you need first.

The client has two more methods about the connection itself. `is_closed` answers immediately from local state: is the connection task still there? `check_connection` (added in 0.7.15) makes one round trip to the server and fails if it does not come back.

```
use tokio_postgres::NoTls;
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;

    // Reported by the server at startup: read them before the connection moves away.
    let version = connection
        .parameter("server_version")
        .unwrap_or("?")
        .to_owned();
    let encoding = connection.parameter("client_encoding").map(str::to_owned);
    let task = tokio::spawn(connection);
    println!("PostgreSQL {version}");
    assert_eq!(encoding.as_deref(), Some("UTF8"));

    client.check_connection().await?; // one round trip, no statement
    assert!(!client.is_closed());

    // End our own session from the server side, as an administrator, a restart or a crash
    // would. The query that triggers it fails with SQLSTATE 57P01, "admin shutdown".
    let error = client
```

```
        .execute("SELECT pg_terminate_backend(pg_backend_pid())", &[])
        .await
        .unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::ADMIN_SHUTDOWN));

    // The connection task ends with an error, and from then on the client knows it is
closed.
    assert!(task.await.expect("the task did not panic").is_err());
    assert!(client.is_closed());
    let error = client.check_connection().await.unwrap_err();
    assert!(error.is_closed());
    Ok(())
}
```

The server reports more than a dozen such parameters. The ones worth knowing: `server_version` (for feature checks), `server_encoding` and `client_encoding` (always `UTF8` for tokio-postgres, which insists on it), `TimeZone`, `DateStyle`, `integer_datetimes`, `application_name`, `is_superuser`, `in_hot_standby` (whether you are on a read-only replica, PG 14+) and, from PG 18, `search_path`. The server also sends an update whenever one of them changes during the session, and `parameter` returns the latest value it has seen.

A connection ends in one of three ways:

How	The connection future returns	The client
You drop every `Client` (and every `Transaction`, which borrows one)	`Ok(())`, after telling the server goodbye	gone
The server ends the session (restart, `pg_terminate_backend`, `idle_session_timeout`) or the network breaks	`Err(…)`	`is_closed()` turns true; every call fails with "connection closed"
You drop the `Connection` future (abort its task)	—	`is_closed()` turns true, and the server sees the socket close

There is no reconnecting: a closed client stays closed. Making a new connection is the caller's job, which is a large part of what a pool does (Chapter 13). Chapter 12 covers how to notice a connection that died silently, without the server telling you.

1.6 What failure looks like

Connecting can fail in several distinct ways, and the error says which, if you know where to look:

```
use std::time::{Duration, Instant};
use tokio_postgres::error::SqlState;
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let good: Config = std::env::var("DATABASE_URL")
        .expect("DATABASE_URL is set")
        .parse()?;

    // The server is there, but says no: there is no such database.
    let mut config = good.clone();
    config.dbname("no_such_database");
    let error = config.connect(NoTls).await.err().expect("refused");
```

```
    assert_eq!(error.code(), Some(&SqlState::INVALID_CATALOG_NAME)); // 3D000
    // The error's own text is terse; the server's message is one level down.
    assert_eq!(error.to_string(), "db error");
    let db = error.as_db_error().expect("an error from the server");
    assert_eq!(db.message(), "database \"no_such_database\" does not exist");

    // Nothing listens on the port: an IO error, and no SQLSTATE.
    let mut config = Config::new();
    config.host("localhost").port(1).user("nobody");
    let error = config.connect(NoTls).await.err().expect("refused");
    assert!(error.code().is_none() && error.as_db_error().is_none());
    let cause = std::error::Error::source(&error).expect("an underlying IO error");
    println!("{error}: {cause}"); // "error connecting to server: Connection refused …"

    // A host that never answers. Without connect_timeout, this waits as long as the
    // operating system lets a TCP connection attempt wait: often over a minute.
    let mut config = Config::new();
    config
        .host("10.255.255.1")
        .user("nobody")
        .connect_timeout(Duration::from_millis(300));
    let started = Instant::now();
    assert!(config.connect(NoTls).await.is_err());
    assert!(started.elapsed() < Duration::from_secs(2));
    Ok(())
}
```

The pattern to remember: **an error from the server has a SQLSTATE code and a `DbError` with the server's message; anything else (refused, unreachable, timed out, TLS) has neither, and its cause is the error's `source()`.** A few codes you will meet when connecting:

SQLSTATE	`SqlState` constant	Meaning
`28P01`	`INVALID_PASSWORD`	Wrong password
`28000`	`INVALID_AUTHORIZATION_SPECIFICATION`	No `pg_hba.conf` entry lets you in (wrong host, user or SSL mode)
`3D000`	`INVALID_CATALOG_NAME`	No such database
`53300`	`TOO_MANY_CONNECTIONS`	`max_connections` reached
`57P03`	`CANNOT_CONNECT_NOW`	The server is starting up or shutting down

> **Pitfall:** `error.to_string()` for any server error is just `db error`. Logging errors with `{}` hides the one thing you need. Log with `{:?}`, or reach for `as_db_error()`; Chapter 6 builds an error type that does it for you.

Side by side – connecting

The task: connect to the book's database as an application called `demo`, and ask for the server's version.

```
-- psql: the connection string is the argument; \conninfo describes the connection
-- (as a table since psql 18).
--   psql "postgres://postgres:secret@localhost:5432/tpbook?application_name=demo"
tpbook=# \conninfo
      Connection Information
      Parameter       |   Value
----------------------+-----------
 Database             | tpbook
```

```
 Client User           | postgres
 Host                  | localhost
 Host Address          | ::1
 Server Port           | 5432
 Options               |
 Protocol Version      | 3.0
 Password Used         | true
 GSSAPI Authenticated | false
 Backend PID           | 95981
 SSL Connection        | false
 Superuser             | on
 Hot Standby           | off
(13 rows)

tpbook=# SHOW server_version;
 server_version
----------------
 18.6
(1 row)
```

```
// sqlx (illustration)
use sqlx::Connection;
let mut conn = sqlx::PgConnection::connect(&url).await?; // no task to spawn
let version: String = sqlx::query_scalar("SHOW server_version").fetch_one(&mut conn).await?;
```

```
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut config: Config = std::env::var("DATABASE_URL")
        .expect("DATABASE_URL is set")
        .parse()?;
    config.application_name("demo");
    let (client, connection) = config.connect(NoTls).await?;
    tokio::spawn(connection);

    let version: String = client.query_one("SHOW server_version", &[]).await?.get(0);
    assert!(version.chars().next().is_some_and(|c| c.is_ascii_digit()));
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	A connection string in the URL or key/value form; one session on the server, with the same startup and login	The same	The same
Different	Built on libpq: reads `PG*` environment variables, `~/.pgpass` and service files	Reads the `PG*` variables too. A `PgConnection` does its IO inside each call, so it needs `&mut` and runs one query at a time; a pool provides concurrency	Reads nothing implicitly. The `Connection` is a separate task, so `Client` methods take `&self`: a client can be shared, and its queries pipelined (Chapter 7). You must spawn the connection

1.7 The playground's `connect`

From here on, the book's examples start with `playground::connect()`, which Chapter 0 asked you to put in your playground's `src/lib.rs`. Now every line of it should make sense:

```rust
use tokio_postgres::{Client, Error, NoTls};

/// Connects to the database named by the `DATABASE_URL` environment variable and spawns the
/// connection task, returning the client.
///
/// # Panics
///
/// If `DATABASE_URL` is not set: examples are programs, not libraries, and a clear panic is
/// the most useful thing they can do about a missing setting.
pub async fn connect() -> Result<Client, Error> {
    let url = std::env::var("DATABASE_URL")
        .expect("set DATABASE_URL, e.g. postgres://postgres:secret@localhost:5432/tpbook");
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;
    tokio::spawn(async move {
        if let Err(e) = connection.await {
            eprintln!("connection error: {e}");
        }
    });
    Ok(client)
}
```

It returns `Result<Client, Error>`, so examples write `connect().await?` in a `main` that returns `Result<(), tokio_postgres::Error>`, and any failure, including at startup, ends the program with the error printed.

In stockroom

stockroom never calls `tokio_postgres::connect` with a bare string. Its settings start from the `--database-url` flag (or `STOCKROOM_DATABASE_URL`) and then get the defaults every session of the service should have, unless the operator's string already says otherwise:

```rust
// stockroom/src/config.rs (excerpt)
pub fn pg_config(&self) -> Result<Config, tokio_postgres::Error> {
    let mut config: Config = self.database_url.parse()?;
    if config.get_application_name().is_none() {
        config.application_name("stockroom");
    }
    if config.get_connect_timeout().is_none() {
        config.connect_timeout(Duration::from_secs(5));
    }
    // Notice a dead peer in about a minute rather than the default two hours.
    config.keepalives_idle(Duration::from_secs(30));
    config.keepalives_interval(Duration::from_secs(10));
    config.keepalives_retries(3);

    // Guard rails on the server: runaway statements, lock waits, and transactions left
    // open by a crashed or stuck client are all ended by PostgreSQL itself.
    let mut options = config.get_options().unwrap_or_default().to_owned();
    // … appends "-c statement_timeout=…", "-c lock_timeout=2000" and
    //   "-c idle_in_transaction_session_timeout=60000" to whatever options were given …
    config.options(options);
    Ok(config)
}
```

The application name is what makes stockroom's sessions recognizable in `pg_stat_activity` (the `/admin/sessions` endpoint shows them). The timeouts and keepalives are Chapter 12's subject. Note the care

taken with `options`: it is a single string, so new settings are appended to the operator's rather than replacing them.

Most of stockroom's connections come from a pool, which spawns each connection and discards its error (Chapter 13). The commands that need one connection of their own (`migrate`, `seed`, `export`) get it from db/connect.rs, which spawns the connection the careful way. Rather than just awaiting it, it *drives* it, so that the server's notices end up in the log:

```
// stockroom/src/db/connect.rs (excerpt)
pub async fn connect(config: &Config, tls: &MakeTls) -> Result<Client,
tokio_postgres::Error> {
    let (client, mut connection) = config.connect(tls.clone()).await?;
    tokio::spawn(async move {
        let mut messages = stream::poll_fn(move |cx| connection.poll_message(cx));
        while let Some(message) = messages.next().await {
            match message {
                // … log notices and notifications …
                Err(error) => {
                    tracing::error!(%error, "database connection failed");
                    break;
                }
            }
        }
    });
    Ok(client)
}
```

`poll_message` is Chapter 11's subject. For now, notice the outline: it is the same spawn-and-log as in §1.2, with a loop where the single `.await` was.

Summary

- `connect` returns a `Client`, which you call, and a `Connection`, a future that does all the IO. Spawn the connection, and log its error: that is where a broken connection shows up first.
- If the connection is not polled, queries wait forever. `connect().await` succeeding means only that the server accepted you.
- `Client` methods take `&self`: one client can be shared by many tasks.
- Connection strings come as key/value pairs or URLs; both parse into a `Config`. tokio-postgres reads no environment variables or password files.
- `Config` setters chain. `options` carries session settings to the server at startup. `host` and `port` *append* to lists.
- `Connection::parameter` reads what the server reported at startup; read it before spawning. `is_closed` is local and free; `check_connection` costs a round trip.
- Errors from the server have a SQLSTATE and a `DbError`; IO errors have neither and keep their cause in `source()`. `Display` of a server error is only "db error".

Exercises

1. **A connection with a name.** Connect with `application_name` set to your own name, then, from the same client, find your session in `pg_stat_activity` and print its `pid`, `application_name` and `backend_start`. *Hint:* `pg_backend_pid()` is your session's pid.
2. **Two clients, one connection?** `Client` is not `Clone`. How would you give ten tasks the same client? Write it, with each task running `SELECT pg_backend_pid()`, and check that all ten see the same pid. *Hint:* `Arc`.
3. **Settings at startup versus `SET`.** Set `statement_timeout` once through `options` and once with `client.batch_execute("SET statement_timeout = 1000")` after connecting. Both work. Name one situation where only the first one is safe. *Hint:* think of what happens to `SET` on a connection that a pool hands to someone else later, and of the connection's very first statement.
4. **What the server reports.** Print every startup parameter listed in §1.5. Then change your session's time zone with `SET TimeZone = 'Asia/Tokyo'` and check whether `connection.parameter("TimeZone")` notices. *Hint:* you need to keep polling the connection yourself to look at it after the `SET`; `select!` from §1.2 can do that in a small program.

Solutions

2.

```rust
use std::sync::Arc;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = Arc::new(playground::connect().await?);
    let mut tasks = Vec::new();
    for _ in 0..10 {
        let client = Arc::clone(&client);
        tasks.push(tokio::spawn(async move {
            let row = client.query_one("SELECT pg_backend_pid()", &[]).await?;
            Ok::<i32, tokio_postgres::Error>(row.get(0))
        }));
    }
    let mut pids = Vec::new();
    for task in tasks {
        pids.push(task.await.expect("no panic")?);
    }
    pids.dedup();
    assert_eq!(pids.len(), 1, "one connection, one server process");
    Ok(())
}
```

All ten requests went through one connection, served by one server process. Chapter 7 shows that they did not wait for each other.

3. A `SET` sticks to the session. On a pooled connection it outlives your use of it and surprises the next borrower, and it is easy to forget to undo it on an error path. Settings in `options` are part of how every connection is made, before any statement runs, so they cannot be skipped, left over or applied late. The other difference is cost: `SET` is one more round trip per connection.

4. It does: the server sends a fresh *ParameterStatus* message when a reported setting changes, and the connection updates its copy. To look at it afterwards you must still own the connection, so poll it by reference, in the same task, while the `SET` runs:

```rust
use tokio_postgres::NoTls;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");
    let (client, mut connection) = tokio_postgres::connect(&url, NoTls).await?;
    let before = connection.parameter("TimeZone").map(str::to_owned);

    // `&mut connection`: polled here, but still ours afterwards.
    tokio::select! {
        result = client.batch_execute("SET TimeZone = 'Asia/Tokyo'") => result?,
        ended = &mut connection => panic!("the connection ended: {ended:?}"),
    }
    assert_eq!(connection.parameter("TimeZone"), Some("Asia/Tokyo"));
    assert_ne!(before.as_deref(), Some("Asia/Tokyo"));
    Ok(())
}
```

The server sends the new value just before it reports that the `SET` is done, so by the time `batch_execute` returns, the connection has already read it.

CHAPTER 2

Running SQL: execute, query and parameters

A `Client` has about twenty methods that run SQL. They look like a long list to learn, but they answer only three questions: do you want rows back, how many, and how should your values travel? This chapter covers the everyday ones: `batch_execute` for scripts, `execute` for changes, `query` and its *one*, *opt* and *scalar* variants for reading, and `simple_query` for when you need to see everything as text.

The most important idea in the chapter is not a method but a rule. **Values never go into the SQL text.** They travel separately, as parameters. The rule makes SQL injection impossible, and tokio-postgres's API is built around it.

2.1 `batch_execute`: scripts

`batch_execute` takes a string with any number of statements, runs them all, and returns nothing but success or failure. It is for schema scripts, migrations and setting things up, and the examples in this chapter use it to create the tables they need:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;

    // Several statements in one string: a table, and some data to start with.
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 sku         text   PRIMARY KEY,
                 name        text   NOT NULL,
                 price_cents bigint NOT NULL CHECK (price_cents > 0)
             );
             INSERT INTO products VALUES ('CHAIR-01', 'Garden chair', 4999);
             INSERT INTO products VALUES ('LAMP-01', 'Desk lamp', 2999);",
        )
        .await?;

    // The statements of one batch run in one implicit transaction: when one fails, the
    // ones before it are undone too.
    let failed = client
        .batch_execute(
            "INSERT INTO products VALUES ('VASE-01', 'Vase', 1999);
             INSERT INTO products VALUES ('RUG-01', 'Rug', 0);", // breaks the CHECK
        )
        .await;
    assert!(failed.is_err());
    let count: i64 = client
        .query_one("SELECT count(*) FROM products", &[])
```

```
        .await?
        .get(0);
    assert_eq!(count, 2, "the vase was rolled back with the rug");
    Ok(())
}
```

`CREATE TEMP TABLE` makes a table that exists only for this session and disappears when the connection closes. That is why the book's examples can run in any order without cleaning up after themselves.

> **SQL lens:** when one message carries several statements, PostgreSQL runs them as a single transaction unless they contain transaction commands of their own (`BEGIN`, `COMMIT`). That is the behaviour above, and it is often what you want from a script: all of it or none of it. One exception: statements that cannot run inside a transaction, such as `CREATE INDEX CONCURRENTLY` and `VACUUM`, cannot run in a multi-statement batch either.

`batch_execute` takes no parameters. That is the price of running several statements at once, and it makes `batch_execute` the wrong tool for anything that includes a value from outside your program. §2.3 explains why, and §2.7 why it is built that way.

2.2 `execute`: changes, counted

`execute` runs one statement and returns how many rows it affected, as a `u64`:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let created = client
        .execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, name text, price_cents
bigint)",
            &[],
        )
        .await?;
    assert_eq!(created, 0, "DDL affects no rows");

    let inserted = client
        .execute(
            "INSERT INTO products VALUES
                 ('CHAIR-01', 'Garden chair', 4999), ('LAMP-01', 'Desk lamp', 2999),
                 ('RUG-01', 'Rug', 8999)",
            &[],
        )
        .await?;
    assert_eq!(inserted, 3);

    // 10% off everything over 30.00: two rows.
    let discounted = client
        .execute(
            "UPDATE products SET price_cents = price_cents * 9 / 10 WHERE price_cents > $1",
            &[&3000_i64],
        )
        .await?;
    assert_eq!(discounted, 2);

    // Deleting something that is not there is not an error: zero rows.
    let deleted = client
        .execute("DELETE FROM products WHERE sku = $1", &[&"VASE-01"])
```

```
        .await?;
    assert_eq!(deleted, 0);
    Ok(())
}
```

The count is what the server reports when the statement completes: rows inserted, updated or deleted, rows returned by a `SELECT`, zero for everything else. The count is often the whole answer. "Was the order cancelled?" is `execute(…) == 1`, and Chapter 8 uses exactly that for optimistic locking.

`&[&3000_i64]` is the first parameter, `$1`. It is time to look at those properly.

2.3 Parameters

A statement refers to its parameters by position, `$1`, `$2` and so on, and the values follow in a slice, in the same order:

```
client.execute(
    "UPDATE products SET price_cents = $2 WHERE sku = $1",
    &[&sku, &new_price],
).await?;
```

The SQL text and the values go to the server **in different protocol messages**. The server parses the text first, with `$1` and `$2` as placeholders, and only then receives the values, as data. A value can contain quotes, semicolons or `DROP TABLE` and remain a value. Here is the difference it makes, with a value a user typed into a form:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE reviews (product text, body text, author text);
             INSERT INTO reviews VALUES ('CHAIR-01', 'Sturdy.', 'Ada'),
                                        ('CHAIR-01', 'Comfy.', 'Bob');",
        )
        .await?;

    // What someone typed into the "your name" field.
    let author = "Eve'); DELETE FROM reviews; --";

    // As a parameter, the text is a value, whatever it contains.
    client
        .execute(
            "INSERT INTO reviews (product, body, author) VALUES ($1, $2, $3)",
            &[&"CHAIR-01", &"Nice.", &author],
        )
        .await?;
    let count: i64 = client
        .query_one("SELECT count(*) FROM reviews", &[])
        .await?
        .get(0);
    assert_eq!(count, 3);

    // Pasted into the SQL text, the text *is* SQL. Never do this.
    let sql = format!(
        "INSERT INTO reviews (product, body, author) VALUES ('CHAIR-01', 'Nice.',
'{author}')"
    );
    client.batch_execute(&sql).await?;
    let count: i64 = client
        .query_one("SELECT count(*) FROM reviews", &[])
```

```
        .await?
        .get(0);
    assert_eq!(count, 0, "every review is gone");
    Ok(())
}
```

The formatted string became three statements: an `INSERT`, a `DELETE FROM reviews`, and a comment that swallowed the rest. Escaping quotes yourself is not the fix; there are more ways to break out of a string literal than quotes. Parameters are the fix.

> **Rust refresher:** the parameter slice has the type `&[&(dyn ToSql + Sync)]`: a slice of references to *trait objects*. Each element can be a different type (`&String`, `&i64`, `&Option<bool>`), as long as it implements `ToSql`, the trait that encodes a Rust value in PostgreSQL's format (Chapters 3 and 5). They are references because the values stay where they are; the slice only points at them. `Sync` is there because the query's future holds these references across `.await`, and for a future to be `Send` (spawnable), everything it borrows must be `Sync`. You rarely write the type out; `&[&a, &b]` coerces to it.

You do write it out when the parameters are not known in advance, for instance a search with optional filters. The SQL text grows one placeholder per filter, and the values go into a `Vec`:

```
use tokio_postgres::Client;
use tokio_postgres::types::ToSql;

/// SKUs of the products matching the filters that are set.
async fn search(
    client: &Client,
    name: Option<&str>,
    max_price: Option<i64>,
) -> Result<Vec<String>, tokio_postgres::Error> {
    let mut sql = String::from("SELECT sku FROM products WHERE true");
    let mut params: Vec<&(dyn ToSql + Sync)> = Vec::new();
    let pattern; // declared out here: `params` borrows it after the `if` ends
    if let Some(name) = name {
        pattern = format!("%{name}%");
        params.push(&pattern);
        sql += &format!(" AND name ILIKE ${}", params.len());
    }
    if let Some(max) = &max_price {
        params.push(max);
        sql += &format!(" AND price_cents <= ${}", params.len());
    }
    sql += " ORDER BY sku";
    let rows = client.query(&sql, &params).await?;
    Ok(rows.iter().map(|row| row.get(0)).collect())
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, name text, price_cents
bigint);
             INSERT INTO products VALUES ('CHAIR-01', 'Garden chair', 4999),
                 ('CHAIR-02', 'Office chair', 12999), ('LAMP-01', 'Desk lamp', 2999);",
        )
        .await?;
```

```
    assert_eq!(search(&client, None, None).await?.len(), 3);
    assert_eq!(
        search(&client, Some("chair"), None).await?,
        ["CHAIR-01", "CHAIR-02"]
    );
    assert_eq!(
        search(&client, Some("chair"), Some(5000)).await?,
        ["CHAIR-01"]
    );
    assert_eq!(search(&client, None, Some(3000)).await?, ["LAMP-01"]);
    Ok(())
}
```

`format!` appears there too, but notice what it inserts: placeholders (`$1`, `$2`), never values. Building SQL *structure* in Rust is fine; putting *data* into it is not.

The number of values must match the number of placeholders. tokio-postgres checks before sending anything, and fails with "expected 2 parameters but got 1".

Identifiers are not values

Parameters stand for values, not for parts of the statement's structure. A table name, a column name or a sort direction cannot be a parameter: `SELECT * FROM $1` is a syntax error (SQLSTATE 42601), because the server must know which table it is planning for before any value arrives. When structure must vary, choose it from a fixed list in Rust:

```
/// The columns a client may sort products by.
fn sort_column(requested: &str) -> Option<&'static str> {
    match requested {
        "sku" => Some("sku"),
        "price" => Some("price_cents"),
        "name" => Some("name"),
        _ => None,
    }
}
```

Only when the name genuinely comes from outside, as with the channel names a user gives pglisten, quote it as an identifier: in double quotes, with any double quote doubled. That is `quote_ident`, ten characters of code and the one place in this repository where outside text enters SQL.

> **Pitfall:** `LIMIT`, `OFFSET`, and values in `WHERE`, `SET` or `VALUES` *can* be parameters; table and column names, `ASC/DESC`, and SQL keywords cannot. If you find yourself writing `format!` with a variable that is not a placeholder number, stop and ask which kind it is.

2.4 `query`, `query_one` and `query_opt`: reading

`query` runs a statement and collects every row into a `Vec<Row>`. Two variants say how many rows you expect, and turn a surprise into an error:

Method	Returns	Fails when
`query`	`Vec<Row>`	never, because of the row count

Method	Returns	Fails when
`query_one`	`Row`	there is not exactly one row
`query_opt`	`Option<Row>`	there is more than one row

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, name text, price_cents
bigint);
             INSERT INTO products VALUES ('CHAIR-01', 'Garden chair', 4999),
                                         ('LAMP-01', 'Desk lamp', 2999);",
        )
        .await?;

    let all = client
        .query("SELECT sku, price_cents FROM products ORDER BY sku", &[])
        .await?;
    assert_eq!(all.len(), 2);
    assert_eq!(all[1].get::<_, &str>("sku"), "LAMP-01");

    let chair = client
        .query_one("SELECT name FROM products WHERE sku = $1", &[&"CHAIR-01"])
        .await?;
    assert_eq!(chair.get::<_, &str>("name"), "Garden chair");

    let vase = client
        .query_opt("SELECT name FROM products WHERE sku = $1", &[&"VASE-01"])
        .await?;
    assert!(vase.is_none());

    // Two rows where one was promised: an error, not a silently ignored second row.
    let error = client
        .query_one("SELECT sku FROM products", &[])
        .await
        .unwrap_err();
    assert_eq!(
        error.to_string(),
        "query returned an unexpected number of rows"
    );
    assert!(
        error.as_db_error().is_none(),
        "the server did nothing wrong"
    );
    Ok(())
}
```

Use `query_one` when a missing row is a bug (a count, a row you just inserted) and `query_opt` when it is an answer ("is there a product with this SKU?"). Mapping "not found" to a 404 starts with choosing `query_opt`. Note that the row-count error is the client's own: there is no SQLSTATE to match on, and Chapter 6 shows how to tell it apart from other errors.

> **Pitfall:** `query_one` does not add `LIMIT 1` to your statement. It reads every row the server sends and then complains if there was more than one. A query that can match many rows when you want the first needs `ORDER BY … LIMIT 1` in the SQL.

2.5 Scalars: `query_scalar`, `query_one_scalar`, `query_opt_scalar`

Many queries return a single column, and taking it out of each row is boilerplate. Since 0.7.16, `Client` has shortcuts that return the first column directly:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999), ('LAMP-01', 2999);",
        )
        .await?;

    let count: i64 = client
        .query_one_scalar("SELECT count(*) FROM products", &[])
        .await?;
    let skus: Vec<String> = client
        .query_scalar("SELECT sku FROM products ORDER BY sku", &[])
        .await?;
    let price: Option<i64> = client
        .query_opt_scalar(
            "SELECT price_cents FROM products WHERE sku = $1",
            &[&"VASE-01"],
        )
        .await?;

    assert_eq!(count, 2);
    assert_eq!(skus, ["CHAIR-01", "LAMP-01"]);
    assert_eq!(price, None);
    Ok(())
}
```

The value type must own its data (`String`, not `&str`), because the row it came from is dropped before the method returns. In 0.7.18 the scalar methods exist only on `Client`, not on `Transaction` or `GenericClient` (Chapter 8), so code that must also run inside a transaction uses `query_one(…)?.get(0)`.

2.6 `RETURNING`: changing and reading in one statement

An `INSERT`, `UPDATE` or `DELETE` can return the rows it touched, with `RETURNING`. Run such a statement with `query` (or `query_one`) instead of `execute`, and you get the generated id, the default timestamp or the new price without a second query:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id          bigint GENERATED ALWAYS AS IDENTITY,
                 sku         text   PRIMARY KEY,
                 price_cents bigint NOT NULL
             )",
        )
        .await?;

    // The id is made by the server; RETURNING hands it back.
    let id: i64 = client
        .query_one(
            "INSERT INTO products (sku, price_cents) VALUES ($1, $2) RETURNING id",
            &[&"CHAIR-01", &4999_i64],
```

```
        )
        .await?
        .get("id");
    assert_eq!(id, 1);

    // An upsert: insert, or update the price if the SKU exists. PG 18's RETURNING old and
    // new say which one happened, and what the price was before.
    let upsert = "INSERT INTO products (sku, price_cents) VALUES ($1, $2)
                  ON CONFLICT (sku) DO UPDATE SET price_cents = EXCLUDED.price_cents
                  RETURNING old.sku IS NULL AS inserted, old.price_cents AS was";
    let row = client.query_one(upsert, &[&"LAMP-01", &2999_i64]).await?;
    assert!(row.get::<_, bool>("inserted"));
    let row = client.query_one(upsert, &[&"CHAIR-01", &3999_i64]).await?;
    assert!(!row.get::<_, bool>("inserted"));
    assert_eq!(row.get::<_, Option<i64>>("was"), Some(4999));
    Ok(())
}
```

`RETURNING old.…` and `new.…` are new in PostgreSQL 18. Before 18, the usual trick to tell an inserted row from an updated one in an upsert is `RETURNING (xmax = 0) AS inserted`, which relies on an implementation detail of how PostgreSQL marks row versions. It works, and it is worth replacing when you can.

2.7 The simple query protocol

Every method so far uses PostgreSQL's *extended* query protocol: the statement and its values travel in separate messages, and results come back in binary, typed. PostgreSQL has a second, older protocol, the *simple* one, in which a single message carries one or more statements as text, with no parameters, and every value comes back as text. `batch_execute` uses it, which is why it can run several statements and cannot take parameters.

`simple_query` uses it too, and returns everything the server sent back:

```
use tokio_postgres::SimpleQueryMessage;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999), ('LAMP-01', NULL);",
        )
        .await?;

    let messages = client
        .simple_query(
            "SELECT sku, price_cents FROM products ORDER BY sku;
             UPDATE products SET price_cents = 1 WHERE price_cents IS NULL",
        )
        .await?;

    let mut rows = Vec::new();
    let mut completed = Vec::new();
    for message in messages {
        match message {
            SimpleQueryMessage::RowDescription(columns) => {
                assert_eq!(columns[1].name(), "price_cents");
            }
            // Every value is `Option<&str>`: text, or None for NULL. No types.
            SimpleQueryMessage::Row(row) => {
```

```
                rows.push((row.get(0).map(str::to_owned), row.get(1).map(str::to_owned)));
            }
            // One per statement: rows returned or affected.
            SimpleQueryMessage::CommandComplete(count) => completed.push(count),
            _ => {} // the enum is non-exhaustive
        }
    }
    assert_eq!(rows[0], (Some("CHAIR-01".into()), Some("4999".into())));
    assert_eq!(rows[1], (Some("LAMP-01".into()), None));
    assert_eq!(completed, [2, 1]);
    Ok(())
}
```

`SimpleQueryMessage` has three variants. `RowDescription` (since 0.7.11) arrives before the rows of each statement that returns rows, with the column names. Without it, a `SELECT` that matched nothing would leave you no way to print its column headers. `Row` holds one row of text. `CommandComplete` carries a statement's count. It carries only the count: the command tag the server sends ("UPDATE 1", "CREATE TABLE") is not exposed, which is why the tiny psql in this repository prints "OK, 0 affected" where psql prints "CREATE TABLE".

When would you use it?

- **Tools that run SQL they did not write**: a migration runner, an admin console, a psql of your own. They need several statements per string and do not know the result types in advance; text suits them. pgq is built on it.
- **Commands that do not take parameters anyway**: `SET`, `LISTEN`, `VACUUM`, DDL.
- **Some connection poolers.** PgBouncer in its older modes could not carry the extended protocol's named statements across transactions, and the simple protocol always works (Chapters 4 and 13).

For everything with values from outside, use the extended protocol, that is, every method in §2.2 to §2.6.

> **Protocol lens:** the simple protocol is one round trip: a *Query* message with the text, answered by a *RowDescription*, *DataRow*s and a *CommandComplete* for each statement, then *ReadyForQuery*. The extended protocol splits the work into *Parse* (the text, with placeholders), *Bind* (the values), *Describe* and *Execute*, which is what makes parameters and binary values possible, and it needs one round trip more unless the statement was prepared before. Chapter 4 counts them.

Side by side – inserting a row and reading it back

The task: insert a product given as three values, and get back what was stored.

```
-- psql: plain SQL with literals...
tpbook=# INSERT INTO products (sku, name, price_cents) VALUES ('VASE-01', 'Vase', 1999)
tpbook-#   RETURNING sku, price_cents;
   sku    | price_cents
----------+-------------
 VASE-01 |        1999
(1 row)

INSERT 0 1
-- ...or, since psql 16, real parameters with \bind (the extended protocol):
```

```
tpbook=# INSERT INTO products (sku, name, price_cents) VALUES ($1, $2, $3)
tpbook-#   RETURNING sku, price_cents \bind 'RUG-01' 'Rug' 8999 \g
```

```
// sqlx (illustration)
let (sku, price): (String, i64) = sqlx::query_as(
    "INSERT INTO products (sku, name, price_cents) VALUES ($1, $2, $3) RETURNING sku,
price_cents",
)
.bind("VASE-01")
.bind("Vase")
.bind(1999_i64)
.fetch_one(&pool)
.await?;
```

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, name text, price_cents
bigint)",
        )
        .await?;
    let row = client
        .query_one(
            "INSERT INTO products (sku, name, price_cents) VALUES ($1, $2, $3)
             RETURNING sku, price_cents",
            &[&"VASE-01", &"Vase", &1999_i64],
        )
        .await?;
    let (sku, price): (&str, i64) = (row.get("sku"), row.get("price_cents"));
    assert_eq!((sku, price), ("VASE-01", 1999));
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	`$n` placeholders and values sent separately (with `\bind`)	`$n` placeholders, values sent separately	`$n` placeholders, values sent separately
Different	Without `\bind`, values are literals in the text; psql variables (`:'name'`) are quoted and pasted in by psql itself	Values attached one by one with `.bind()`. The `query!` macros check the SQL and types at compile time, against a live database	Values in a slice of references, checked at run time against the types the server reports for the statement. Rows are read by name or position with typed `get` (Chapter 3)

In stockroom

Every repository function in stockroom follows the same pattern: the SQL written out in full, the values as parameters, `RETURNING` instead of a second query. Creating a store:

```
// stockroom/src/repo/stores.rs (excerpt)
pub async fn create(db: &impl GenericClient, new: &NewStore) -> Result<Store, AppError> {
    let stmt = db
        .prepare_cached(
            "INSERT INTO stores (code, name, address) VALUES ($1, $2, $3)
             RETURNING id, code, name, address, created_at",
```

```
        )
        .await?;
    let row = db
        .query_one(&stmt, &[&new.code, &new.name, &new.address])
        .await?;
    Ok(Store::try_from(&row)?)
}
```

Three things in it come later. `prepare_cached` is Chapter 4 and 13 (preparing a statement once per connection). `&new.address` is a Rust struct passed as a PostgreSQL composite value (Chapter 5). `GenericClient` lets the same function run on a pooled connection or inside a transaction (Chapter 8).

The one place stockroom uses the simple protocol is a diagnostic: `GET /admin/sessions` lists the database's sessions straight from `pg_stat_activity`, and only displays them, so text is exactly what it wants:

```
// stockroom/src/repo/reports.rs (excerpt)
let messages = client
    .simple_query(
        "SELECT pid, application_name, backend_type, state,
                wait_event_type || ':' || wait_event AS waiting_on,
                date_trunc('second', now() - query_start) AS running_for,
                left(query, 120) AS query
         FROM pg_stat_activity
         WHERE datname = current_database()
         ORDER BY pid",
    )
    .await?;
let mut sessions = Vec::new();
for message in messages {
    if let SimpleQueryMessage::Row(row) = message {
        let mut session = serde_json::Map::new();
        for (i, column) in row.columns().iter().enumerate() {
            // `Option<&str>`: NULL is `None`; everything else is text, whatever its SQL
type.
            session.insert(column.name().to_owned(), row.get(i).into());
        }
        sessions.push(session);
    }
}
```

The JSON this produces has `"pid": "60027"`, a string, where a typed query would give a number: that is the simple protocol's trade.

Summary

- `batch_execute` runs a script of several statements in one implicit transaction. No parameters, no results.
- `execute` returns the number of rows affected (or returned); 0 for DDL.
- `query` returns all rows; `query_one` exactly one; `query_opt` zero or one. A wrong row count is an error from the client, with no SQLSTATE.
- `query_scalar`, `query_one_scalar` and `query_opt_scalar` (0.7.16+, `Client` only) return the first column of each row.
- Values go in parameters, `$1…$n`, never into the SQL text. Building SQL *structure* with `format!` is fine; table names, column names and keywords come from allowlists, or `quote_ident` as a last resort.

- A dynamic parameter list is a `Vec<&(dyn ToSql + Sync)>`, with the values it points at living at least as long.
- `RETURNING` reads what a change produced. PG 18's `RETURNING old/new` shows before and after.
- `simple_query` uses the simple protocol: several statements, no parameters, all text, with `RowDescription`, `Row` and `CommandComplete` messages.

Exercises

1. **A price change.** Write `async fn reprice(client: &Client, percent: i64, min_price: i64) -> Result<u64, Error>` that raises every price at or above `min_price` by `percent` percent and returns how many products changed. *Hint:* integer arithmetic in SQL: `price_cents * (100 + $1) / 100`.
2. **One more filter.** Extend `search` from §2.3 with `min_price: Option<i64>`. Make sure the placeholder numbers stay right in every combination of filters. *Hint:* the number of each placeholder is `params.len()` right after the push.
3. **Spot the injection.** A colleague wrote this for a "sort by" dropdown. What can a client do with it, and how do you fix it?

```
let direction = query_params.get("dir").unwrap_or("asc");
let sql = format!("SELECT sku FROM products ORDER BY price_cents {direction}");
client.query(&sql, &[]).await?
```

4. **Found, or not.** Write `async fn price_of(client: &Client, sku: &str) -> Result<Option<i64>, Error>`, which returns `Ok(None)` for an unknown SKU and an error only when something actually went wrong. *Hint:* one of the scalar methods does exactly this.
5. **A tiny psql.** Use `simple_query` to run a script typed by a user and print, for each statement, either its rows or "OK, n affected". Compare your version with `run` in apps/pgq/src/lib.rs.

Solutions

3. `direction` is pasted into the SQL, so a client can send anything: `?dir=asc; DELETE FROM products` does not run here only because `query` uses the extended protocol, which allows one statement per message (the server refuses with "cannot insert multiple commands into a prepared statement"). But `?dir=(SELECT …)` can still add subqueries to the sort, and a later refactoring to `batch_execute` would make the first attack work too. A sort direction is structure, not a value, so it cannot be a parameter. Map it from an allowlist:

```
let direction = match query_params.get("dir") {
    Some("desc") => "DESC",
    _ => "ASC",
};
let sql = format!("SELECT sku FROM products ORDER BY price_cents {direction}");
```

Now `direction` is always one of two string literals in your binary.

4.

```
use tokio_postgres::{Client, Error};

async fn price_of(client: &Client, sku: &str) -> Result<Option<i64>, Error> {
```

```
    client
        .query_opt_scalar("SELECT price_cents FROM products WHERE sku = $1", &[&sku])
        .await
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999);",
        )
        .await?;
    assert_eq!(price_of(&client, "CHAIR-01").await?, Some(4999));
    assert_eq!(price_of(&client, "NOPE-01").await?, None);
    Ok(())
}
```

`query_opt_scalar` returns `Ok(None)` for no row and an error for anything else, including two rows (impossible here, because `sku` is the primary key).

CHAPTER 3

Rows and the type mapping

A query hands you `Rows`, and every value in them must become a Rust value. tokio-postgres does that with one rule: **you say which Rust type you want, and the conversion checks that the column's PostgreSQL type fits.** There is no guessing and no silent widening, and a mismatch is an error at run time, so the mapping between the two type systems is something you need to know.

This chapter covers reading a `Row`, the full table of which Rust types match which PostgreSQL types (including the ones optional cargo features add), NULL, borrowing values without copying them, and the types that trip people up: timestamps, JSON, arrays and exact decimals. Chapter 5 does the same for types you define yourself.

3.1 Reading a `Row`

A `Row` knows its columns, as the server described them, and hands out values by position or by name:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let row = client
        .query_one(
            "SELECT 'CHAIR-01' AS sku, 4999::int8 AS price_cents, NULL::text AS note",
            &[],
        )
        .await?;

    // What the row knows: the names and types the server described.
    let described: Vec<(&str, &str)> = row
        .columns()
        .iter()
        .map(|c| (c.name(), c.type_().name()))
        .collect();
    assert_eq!(
        described,
        [("sku", "text"), ("price_cents", "int8"), ("note", "text")]
    );
    assert_eq!(row.len(), 3);

    // By position or by name. The Rust type you ask for decides the conversion.
    let sku: &str = row.get(0);
    let price: i64 = row.get("price_cents");
    assert_eq!((sku, price), ("CHAIR-01", 4999));

    // `get` panics on any problem; `try_get` returns it.
    let wrong_type = row.try_get::<_, i32>("price_cents").unwrap_err();
    let no_column = row.try_get::<_, i64>("prise_cents").unwrap_err();
    let was_null = row.try_get::<_, String>("note").unwrap_err();
    assert_eq!(wrong_type.to_string(), "error deserializing column 1");
    assert_eq!(no_column.to_string(), "invalid column `prise_cents`");
```

```
    assert_eq!(was_null.to_string(), "error deserializing column 2");

    // NULL becomes None, if you ask for an Option.
    let note: Option<String> = row.get("note");
    assert_eq!(note, None);
    Ok(())
}
```

Three kinds of failure, then, all at run time:

Failure	`Debug` of the error	Cause
Wrong type	`Error { kind: FromSql(1), cause: Some(WrongType { postgres: Int8, rust: "i32" }) }`	The Rust type does not accept the column's type
No such column	`Error { kind: Column("prise_cents"), cause: None }`	A name that is not in the row, or an index past the end
NULL into a non-`Option`	`Error { kind: FromSql(2), cause: Some(WasNull) }`	The value was NULL

Note how little the `Display` text says ("error deserializing column 1"): the useful part is the cause, in `source()`. Chapter 6 makes that visible.

`get` or `try_get`? `get` is convenient where a failure is a bug you want to hear about loudly: examples, tests, and columns of a query you wrote right above. In code that serves requests, prefer `try_get` with `?`: a panic in a handler takes down a request, or worse, and the error has more to say than the panic message.

By name or by position? Positions are fast and break silently when someone adds a column to the `SELECT`. Names survive that, and cost a scan of the column list (a handful of string comparisons). In application code, use names. If two columns have the same name, which happens with `SELECT *` over a join, the first one wins. Give them distinct names with `AS`.

3.2 The type table

Every Rust type that can be read from a row implements `FromSql`, and every type that can be sent as a parameter implements `ToSql` (both traits come from the `postgres-types` crate and are re-exported as `tokio_postgres::types`). Each implementation says which PostgreSQL types it accepts. Built in, with no features:

Rust	PostgreSQL
`bool`	`boolean`
`i8`	`"char"` (the one-byte internal type, not `char(n)`)
`i16`	`smallint` (`int2`)
`i32`	`integer` (`int4`)
`i64`	`bigint` (`int8`)
`u32`	`oid` (and only `oid`)
`f32`	`real` (`float4`)

Rust	PostgreSQL
`f64`	`double precision` (`float8`)
`&str`, `String`, `Box<str>`, `Cow<str>`	`text`, `varchar`, `char(n)` (`bpchar`), `name`, `unknown`, and `citext`, `ltree`, `lquery`, `ltxtquery` by name
`&[u8]`, `Vec<u8>`	`bytea`
`HashMap<String, Option<String>>`	`hstore`
`std::time::SystemTime`	`timestamp`, `timestamptz`
`std::net::IpAddr`	`inet`
`Option<T>`	whatever `T` takes, or NULL
`Vec<T>`, `&[T]`, `Box<[T]>`	arrays of whatever `T` takes (one dimension)
`[T; N]` (feature `array-impls`)	arrays of exactly `N` elements

Everything else comes with an optional feature of `tokio-postgres`, so that you only compile the integrations you use:

Feature	Rust	PostgreSQL
`with-jiff-0_2`	`jiff::Timestamp` / `civil::DateTime` / `civil::Date` / `civil::Time`	`timestamptz` / `timestamp` / `date` / `time`
`with-chrono-0_4`	`DateTime<Utc>`, `DateTime<Local>`, `DateTime<FixedOffset>` / `NaiveDateTime` / `NaiveDate` / `NaiveTime`	`timestamptz` / `timestamp` / `date` / `time`
`with-time-0_3`	`OffsetDateTime` / `PrimitiveDateTime` / `Date` / `Time`	`timestamptz` / `timestamp` / `date` / `time`
`with-uuid-1`	`uuid::Uuid`	`uuid`
`with-serde_json-1`	`serde_json::Value`, `Json<T>`	`json`, `jsonb`
`with-cidr-0_3`	`IpCidr` / `IpInet`	`cidr` / `inet`
`with-eui48-1`	`MacAddress`	`macaddr`
`with-geo-types-0_7`	`Point<f64>` / `Rect<f64>` / `LineString<f64>`	`point` / `box` / `path`
`with-bit-vec-0_8`	`BitVec`	`bit`, `varbit`

And one from outside: `rust_decimal::Decimal` implements both traits for `numeric` when you enable `rust_decimal`'s own feature `db-tokio-postgres` (§3.8).

Three gaps are worth knowing before you hit them:

- **No unsigned integers** except `u32` for `oid`. PostgreSQL has no unsigned types, and pretending would hide overflows. A `u64` count goes into a `bigint` as an `i64`, with `i64::try_from` at the boundary.
- **No `interval`.** None of the date-time features maps it. Convert in SQL (`extract(epoch FROM …)` gives seconds as `numeric`), or write the conversion yourself (Chapter 5).
- **No `money`**, and you do not want it: store cents in a `bigint`, or use `numeric`.

> **Pitfall:** SQL's own typing decides the column type, and it is not always the one you expect. `6 * 7` is `integer`, but `count(*)` is `bigint`, `sum()` of an `integer` column is `bigint`, `sum()` of a `bigint` column is **`numeric`**, and `avg()` of any integer is `numeric` too. `let total: i64 = row.get("sum")` over a `bigint` column fails. Cast in the query (`sum(total_cents)::bigint`) or read a `Decimal`. When in doubt, ask the row: `row.columns()[0].type_()`, or `pgq --describe`.

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let row = client
        .query_one(
            "SELECT count(*), sum(i), sum(i::int8), avg(i), sum(i::int8)::int8 AS total
             FROM generate_series(1, 4) AS i",
            &[],
        )
        .await?;
    let types: Vec<&str> = row.columns().iter().map(|c| c.type_().name()).collect();
    assert_eq!(types, ["int8", "int8", "numeric", "numeric", "int8"]);
    assert_eq!(row.get::<_, i64>("total"), 10);
    Ok(())
}
```

3.3 NULL

NULL is `Option` on both sides. Reading, ask for `Option<T>` wherever a column can be NULL: outer joins, nullable columns, aggregates over no rows (`max()` of nothing is NULL). Sending, pass an `Option`. A bare `None` has no type, so name it: `&None::<i64>`.

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE notes (id int, note text);
             INSERT INTO notes VALUES (1, NULL), (2, 'fragile');",
        )
        .await?;

    // A NULL parameter.
    client
        .execute("INSERT INTO notes VALUES ($1, $2)", &[&3, &None::<&str>])
        .await?;

    // `= $1` with NULL matches nothing: in SQL, NULL = NULL is not true but NULL.
    let no_note: Option<&str> = None;
    let equal = client
        .query("SELECT id FROM notes WHERE note = $1", &[&no_note])
        .await?;
    assert!(equal.is_empty());

    // IS NOT DISTINCT FROM treats two NULLs as equal: the comparison you meant.
    let same: Vec<i32> = client
        .query(
            "SELECT id FROM notes WHERE note IS NOT DISTINCT FROM $1 ORDER BY id",
            &[&no_note],
        )
        .await?
        .iter()
        .map(|row| row.get("id"))
```

```
        .collect();
    assert_eq!(same, [1, 3]);

    // max() over no rows is NULL, even though the column type is integer.
    let max: Option<i32> = client
        .query_one("SELECT max(id) FROM notes WHERE false", &[])
        .await?
        .get(0);
    assert_eq!(max, None);
    Ok(())
}
```

> **SQL lens:** an optional filter ("this note, or any note if none was given") is often written `WHERE $1::text IS NULL OR note = $1`. It works, but a prepared statement with that shape cannot use an index on `note` well, because the plan must suit both cases. When it matters, build two statements, as `search` did in Chapter 2.

3.4 Borrowing from a row

`FromSql` has a lifetime: `FromSql<'a>`, where `'a` is the lifetime of the row's data. Types like `&str` and `&[u8]` take advantage of it. They point into the row's buffer instead of copying, which makes reading many rows cheaper. The price is that they cannot outlive the row:

```
// Does not compile: the row is a temporary, gone at the end of the statement.
let name: &str = client.query_one("SELECT 'chair'", &[]).await?.get(0);
println!("{name}");
// error[E0716]: temporary value dropped while borrowed
```

Keep the row (or the `Vec<Row>`) alive for as long as you use what you borrowed, or ask for an owned type (`String`, `Vec<u8>`) instead:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let rows = client
        .query(
            "SELECT 'Product ' || i AS name FROM generate_series(1, 1000) AS i",
            &[],
        )
        .await?;

    // Borrowed: no allocation per row. `rows` outlives every `&str`.
    let total_length: usize = rows
        .iter()
        .map(|row| row.get::<_, &str>("name").len())
        .sum();
    assert_eq!(
        total_length,
        1000 * "Product ".len() + 9 + 2 * 90 + 3 * 900 + 4
    );

    // Owned: each value has its own allocation, and can go anywhere.
    let names: Vec<String> = rows.iter().map(|row| row.get("name")).collect();
    drop(rows);
    assert_eq!(names[999], "Product 1000");
    Ok(())
}
```

> **Rust refresher:** `FromSql<'a>` with `fn from_sql(ty: &Type, raw: &'a [u8]) -> Result<Self, …>` lets `Self` borrow from `raw`, the column's bytes inside the row. A type that never borrows implements it for every `'a`, which is what `FromSqlOwned` means: it is a shorthand for `for<'a> FromSql<'a>`. APIs that drop the row before handing you the value, such as the `query_scalar` family in Chapter 2, require `FromSqlOwned`. That is why `query_one_scalar::<&str, _>` does not compile.

3.5 Rows into structs

Applications want structs, not rows. tokio-postgres has no derive for this, and the usual pattern is a `TryFrom<&Row>` implementation per struct, reading columns by name:

```
use tokio_postgres::Row;

#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    name: String,
    price_cents: i64,
    discontinued: Option<jiff::civil::Date>,
}

impl TryFrom<&Row> for Product {
    type Error = tokio_postgres::Error;

    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            name: row.try_get("name")?,
            price_cents: row.try_get("price_cents")?,
            discontinued: row.try_get("discontinued")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 sku text PRIMARY KEY, name text NOT NULL, price_cents bigint NOT NULL,
                 discontinued date);
             INSERT INTO products VALUES ('CHAIR-01', 'Garden chair', 4999, NULL),
                                         ('STOOL-01', 'Stool', 1999, '2026-06-30');",
        )
        .await?;

    let rows = client
        .query(
            "SELECT sku, name, price_cents, discontinued FROM products ORDER BY sku",
            &[],
        )
        .await?;
    let products = rows
        .iter()
        .map(Product::try_from)
        .collect::<Result<Vec<_>, _>>()?;

    assert_eq!(products[0].discontinued, None);
    assert_eq!(
        products[1],
        Product {
```

```
            sku: "STOOL-01".into(),
            name: "Stool".into(),
            price_cents: 1999,
            discontinued: Some(jiff::civil::date(2026, 6, 30)),
        }
    );
    Ok(())
}
```

It is a few lines more per struct than a derive, and in exchange, the mapping is ordinary code you can read, step through and change: a column renamed in the query, a computed field, a nested struct read from the same row. Keep the `SELECT` list and the `TryFrom` together, in the same module. Chapter 14 shows how stockroom organizes them, and mentions clorinde, which generates both from `.sql` files.

3.6 Dates and times

PostgreSQL has two timestamp types, and they map to two different Rust types:

PostgreSQL	What it stores	jiff	chrono	time
`timestamptz`	an instant: microseconds since 2000-01-01 UTC	`Timestamp`	`DateTime<Utc>`	`OffsetDateTime`
`timestamp`	a wall-clock reading, no time zone	`civil::DateTime`	`NaiveDateTime`	`PrimitiveDateTime`
`date`	a calendar day	`civil::Date`	`NaiveDate`	`Date`
`time`	a time of day	`civil::Time`	`NaiveTime`	`Time`

This book uses jiff, whose `Timestamp` is exactly what `timestamptz` is: an instant, with no time zone attached. Despite its name, `timestamptz` does *not* store a time zone. It stores UTC and converts at the edges. And the edges tokio-postgres uses are binary, so the session's `TimeZone` setting does not affect what you read:

```
use jiff::Timestamp;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("SET TimeZone = 'America/New_York'")
        .await?;

    let row = client
        .query_one(
            "SELECT '2026-11-06 12:00:00+00'::timestamptz AS at,
                    '2026-11-06 12:00:00+00'::timestamptz::text AS as_text",
            &[],
        )
        .await?;
    let at: Timestamp = row.get("at");
    // Binary: the instant itself, whatever the session's time zone.
    assert_eq!(at.to_string(), "2026-11-06T12:00:00Z");
    // Text: rendered in the session's time zone.
    assert_eq!(row.get::<_, &str>("as_text"), "2026-11-06 07:00:00-05");

    // timestamp (without time zone) needs the civil type; Timestamp refuses it.
    let row = client
        .query_one("SELECT '2026-11-06 12:00'::timestamp", &[])
        .await?;
```

```
    assert!(row.try_get::<_, Timestamp>(0).is_err());
    let wall: jiff::civil::DateTime = row.get(0);
    assert_eq!(wall, jiff::civil::date(2026, 11, 6).at(12, 0, 0, 0));
    Ok(())
}
```

The session time zone still matters for anything the *server* computes from a `timestamptz`: `date_trunc('day', created_at)` and `created_at::date` cut the day at midnight in the session's time zone. stockroom's sales report says which one it means with `AT TIME ZONE 'UTC'`.

> **Pitfall:** PostgreSQL keeps microseconds; jiff, chrono and `time` keep nanoseconds. A value with sub-microsecond digits loses them on the way in, so it does not compare equal on the way back. `Timestamp::now()` has nanoseconds on Linux (on macOS the clock happens to stop at microseconds, which hides the problem until the tests run in CI). Truncate before you store, or compare at microsecond precision.

```
use jiff::Timestamp;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let precise = Timestamp::from_nanosecond(1_700_000_000_123_456_789).expect("in range");
    let stored: Timestamp = client
        .query_one("SELECT $1::timestamptz", &[&precise])
        .await?
        .get(0);
    assert_ne!(stored, precise);
    assert_eq!(stored.to_string(), "2023-11-14T22:13:20.123456Z");
    let truncated = Timestamp::from_microsecond(precise.as_microsecond()).expect("in
range");
    assert_eq!(stored, truncated);
    Ok(())
}
```

3.7 JSON

With `with-serde_json-1`, `serde_json::Value` reads and writes `json` and `jsonb`. For a column with a known shape, `Json<T>` does the same through serde with your own type: it wraps any `T: Serialize` going in and any `T: Deserialize` coming out.

```
use serde::{Deserialize, Serialize};
use serde_json::{Value, json};
use tokio_postgres::types::Json;

#[derive(Debug, PartialEq, Serialize, Deserialize)]
struct Customer {
    name: String,
    email: String,
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE orders (id int, customer jsonb, extra jsonb)")
```

```
        .await?;

    let ada = Customer {
        name: "Ada".into(),
        email: "ada@example.com".into(),
    };
    // A typed struct and a free-form Value, both into jsonb.
    client
        .execute(
            "INSERT INTO orders VALUES (1, $1, $2)",
            &[
                &Json(&ada),
                &json!({ "giftWrap": true, "notes": ["fragile"] }),
            ],
        )
        .await?;

    let row = client
        .query_one("SELECT customer, extra FROM orders WHERE id = 1", &[])
        .await?;
    let Json(customer): Json<Customer> = row.get("customer");
    let extra: Value = row.get("extra");
    assert_eq!(customer, ada);
    assert_eq!(extra["notes"][0], "fragile");

    // Inside the document, in SQL: ->> yields text, -> yields jsonb.
    let row = client
        .query_one(
            "SELECT customer ->> 'name' AS name, extra -> 'giftWrap' AS wrap
             FROM orders WHERE customer ->> 'email' = $1",
            &[&"ada@example.com"],
        )
        .await?;
    assert_eq!(row.get::<_, &str>("name"), "Ada");
    assert_eq!(row.get::<_, Value>("wrap"), json!(true));
    Ok(())
}
```

`json` stores the text as given; `jsonb` stores a parsed form, which is what you want for anything you query, compare or index. The Rust side works the same with both.

> **Pitfall:** `Json<T>` reads whatever is in the column and deserializes it with serde. If the document does not match `T`, for example after someone else's code wrote a slightly different shape, reading fails with a conversion error at run time. For columns written by several programs, read a `Value` and validate it, or give `T` serde defaults.

3.8 Arrays

A one-dimensional PostgreSQL array maps to `Vec<T>` (or a slice going in), for any element type `T` that maps:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE products (sku text PRIMARY KEY, tags text[])")
        .await?;
    for (sku, tags) in [
        ("CHAIR-01", vec!["garden", "sale"]),
```

```
        ("LAMP-01", vec!["lighting"]),
        ("RUG-01", vec![]),
    ] {
        client
            .execute("INSERT INTO products VALUES ($1, $2)", &[&sku, &tags])
            .await?;
    }

    let tags: Vec<String> = client
        .query_one("SELECT tags FROM products WHERE sku = 'CHAIR-01'", &[])
        .await?
        .get(0);
    assert_eq!(tags, ["garden", "sale"]);

    // "Is it one of these?" One array parameter and = ANY, not IN.
    let wanted = ["CHAIR-01", "RUG-01", "VASE-01"];
    let found: Vec<String> = client
        .query_scalar(
            "SELECT sku FROM products WHERE sku = ANY($1) ORDER BY sku",
            &[&&wanted[..]],
        )
        .await?;
    assert_eq!(found, ["CHAIR-01", "RUG-01"]);

    // "Does the array contain this?" @> (contains), indexable with GIN.
    let on_sale: Vec<String> = client
        .query_scalar(
            "SELECT sku FROM products WHERE tags @> ARRAY[$1]",
            &[&"sale"],
        )
        .await?;
    assert_eq!(on_sale, ["CHAIR-01"]);

    // Elements can be NULL: then the element type must be an Option.
    let mixed: Vec<Option<i32>> = client
        .query_one("SELECT ARRAY[1, NULL, 3]", &[])
        .await?
        .get(0);
    assert_eq!(mixed, [Some(1), None, Some(3)]);
    Ok(())
}
```

`= ANY($1)` with one array parameter replaces `IN ($1, $2, …)`. It is one statement whatever the list's length, so it can be prepared once and reused, and the list cannot grow into something you have to build with `format!`.

Two limits: an array with NULL elements needs `Vec<Option<T>>` (with `Vec<T>` it fails with `WasNull`), and arrays of more than one dimension are refused ("array contains too many dimensions"). Model a grid as a table, or as `jsonb`.

> **Pitfall:** `WHERE sku IN ($1)` does not mean "any of the values in this array". `IN` with one item is `sku = $1`, so the server types `$1` as `text`, and the client then refuses to send a `Vec` as a `text`: "cannot convert between the Rust type `Vec<&str>` and the Postgres type `text`". The `ANY` form is the one you want.

3.9 Exact numbers, UUIDs

`numeric` is PostgreSQL's arbitrary-precision decimal. `f64` cannot hold it exactly, and tokio-postgres does not pretend: there is no built-in mapping. `rust_decimal::Decimal` provides one (with its `db-tokio-postgres` feature, in the playground's `Cargo.toml`):

```
use rust_decimal::Decimal;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let row = client
        .query_one(
            "SELECT 0.1::numeric + 0.2 AS exact, 0.1::float8 + 0.2 AS binary,
                    round(avg(x), 2) AS average
             FROM (VALUES (1999), (2999), (4999)) AS prices(x)",
            &[],
        )
        .await?;
    let exact: Decimal = row.get("exact");
    let binary: f64 = row.get("binary");
    let average: Decimal = row.get("average");
    assert_eq!(exact.to_string(), "0.3");
    assert_eq!(binary.to_string(), "0.30000000000000004");
    assert_eq!(average.to_string(), "3332.33");
    Ok(())
}
```

For money that a program adds up, the simplest exact representation is an integer number of cents in a `bigint`. That is what stockroom does, with `numeric` only where SQL produces one (averages).

`uuid` maps to `uuid::Uuid` with `with-uuid-1`. PostgreSQL 18 can generate time-ordered version-7 UUIDs itself, with `uuidv7()`, which makes a good default for primary keys: unique without coordination, and roughly in insertion order, which keeps B-tree indexes compact.

```
use uuid::Uuid;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let row = client
        .query_one("SELECT uuidv7() AS first, uuidv7() AS second", &[])
        .await?;
    let (first, second): (Uuid, Uuid) = (row.get("first"), row.get("second"));
    assert_eq!(first.get_version_num(), 7);
    assert!(first < second, "time-ordered");

    // And from Rust, generated client-side, the same type goes back in.
    let mine = Uuid::now_v7();
    let echoed: Uuid = client.query_one("SELECT $1::uuid", &[&mine]).await?.get(0);
    assert_eq!(echoed, mine);
    Ok(())
}
```

Side by side – a row into a struct

The task: read a product into a struct, with a nullable column.

```
-- psql: rows are printed; \gx shows one column per line, which is how a struct reads.
tpbook=# SELECT sku, name, price_cents, discontinued FROM products WHERE sku = 'STOOL-01'
\gx
-[ RECORD 1 ]+-----------
sku          | STOOL-01
```

```
name         | Stool
price_cents  | 1999
discontinued | 2026-06-30
```

```
// sqlx (illustration)
#[derive(sqlx::FromRow)]
struct Product { sku: String, name: String, price_cents: i64, discontinued:
Option<chrono::NaiveDate> }

let product: Product = sqlx::query_as("SELECT sku, name, price_cents, discontinued FROM
products WHERE sku = $1")
    .bind("STOOL-01")
    .fetch_one(&pool)
    .await?;
```

```
use tokio_postgres::Row;

#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    name: String,
    price_cents: i64,
    discontinued: Option<jiff::civil::Date>,
}

impl TryFrom<&Row> for Product {
    type Error = tokio_postgres::Error;
    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            name: row.try_get("name")?,
            price_cents: row.try_get("price_cents")?,
            discontinued: row.try_get("discontinued")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let row = client
        .query_one(
            "SELECT 'STOOL-01' AS sku, 'Stool' AS name, 1999::int8 AS price_cents,
                    '2026-06-30'::date AS discontinued",
            &[],
        )
        .await?;
    let product = Product::try_from(&row)?;
    assert_eq!(product.discontinued, Some(jiff::civil::date(2026, 6, 30)));
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	Columns have names and types; NULL is a value of its own	NULL is `Option`; types must match	NULL is `Option`; types must match
Different	Everything is displayed as text	A derive maps columns to fields by name; `query_as!` checks the mapping at compile time	No derive: a `TryFrom<&Row>` you write, checked at run time. Values can borrow from the row

In stockroom

Every type stockroom reads from the database has a `TryFrom<&Row>`, next to the queries that produce its columns. The catalog's `Product` mixes a domain type (`Sku`, Chapter 5), money in cents, an array, free-form JSON and two timestamps:

```
// stockroom/src/repo/products.rs (excerpt)
impl TryFrom<&Row> for Product {
    type Error = tokio_postgres::Error;

    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok(Product {
            id: row.try_get("id")?,
            sku: row.try_get("sku")?,
            name: row.try_get("name")?,
            price_cents: row.try_get("price_cents")?,
            tags: row.try_get("tags")?,
            attributes: row.try_get("attributes")?,
            version: row.try_get("version")?,
            created_at: row.try_get("created_at")?,
            updated_at: row.try_get("updated_at")?,
        })
    }
}
```

`id` is a `Uuid` (generated by `uuidv7()`), `tags` a `Vec<String>`, `attributes` a `serde_json::Value`, and the timestamps are jiff `Timestamps`. One generic helper turns a page of rows into structs for every such type, converting the error on the way:

```
// stockroom/src/repo/mod.rs (excerpt)
pub fn from_rows<T>(rows: &[Row]) -> Result<Vec<T>, AppError>
where
    for<'r> T: TryFrom<&'r Row, Error = tokio_postgres::Error>,
{
    rows.iter()
        .map(|row| T::try_from(row).map_err(AppError::from))
        .collect()
}
```

The `for<'r>` says "for a row borrowed for any lifetime", which is what lets `T` be built from each row in turn. An order's customer is structured JSON, read through serde:

```
// stockroom/src/repo/orders.rs (excerpt)
let Json(customer): Json<Customer> = head.try_get("customer")?;
```

Summary

- `row.get` panics and `row.try_get` returns an error, by index or by name. Prefer names, and `try_get` in code that serves requests.
- You choose the Rust type; the conversion checks it against the column's type. Mismatches, missing columns and unexpected NULLs are run-time errors whose cause is in `source()`.
- Know the table: `int4` is `i32`, `int8` is `i64`; no unsigned types except `oid`; `sum(bigint)` and `avg()` are `numeric`.
- NULL is `Option` both ways; name the type of a bare `None`. `= NULL` matches nothing; use `IS NOT DISTINCT FROM`.

- `&str` and `&[u8]` borrow from the row; owned types do not. `FromSqlOwned` means "never borrows".
- Map rows to structs with `TryFrom<&Row>`, reading columns by name.
- `timestamptz` is an instant (jiff `Timestamp`), `timestamp` a wall-clock reading (`civil::DateTime`). Binary transfer ignores the session time zone, and microseconds are the limit.
- JSON: `serde_json::Value`, or `Json<T>` with serde. Arrays: one dimension, `Vec<Option<T>>` for NULL elements, `= ANY($1)` instead of `IN`.
- `numeric` needs `rust_decimal`; `uuid` needs `with-uuid-1`; `interval` has no mapping.

Exercises

1. **A report row.** Write a struct `StoreSales { store: String, orders: i64, revenue_cents: i64 }` and fill it from `SELECT store, count(*) AS orders, sum(total_cents) AS revenue_cents FROM orders GROUP BY store` over a temporary `orders (store text, total_cents bigint)` table. Run it, read the error, and fix it two different ways. *Hint:* §3.2's pitfall.
2. **Zero copies.** Over `SELECT md5(i::text) AS hash FROM generate_series(1, 10000) AS i`, count how many hashes start with `"00"` without allocating a `String` per row. *Hint:* `&str`, and keep the rows alive.
3. **A left join.** Products and their reviews: `products (sku)` and `reviews (sku, stars int)`. Return every product with its average number of stars, or `None` when it has no reviews. Which Rust type do you read the average as?
4. **Find by tags.** Write a query that returns the products having *all* of a list of tags, and one that returns the products having *any* of them. Both take one array parameter. *Hint:* `@>` and `&&`.

Solutions

1. The error is `WrongType { postgres: Numeric, rust: "i64" }`: `sum()` of a `bigint` column is `numeric`, because a sum of many `bigint`s can overflow `bigint`. Either cast in the query, `sum(total_cents)::bigint AS revenue_cents` (which turns an overflow into an error on the server), or read a `rust_decimal::Decimal` and convert it in Rust.

```
#[derive(Debug, PartialEq)]
struct StoreSales {
    store: String,
    orders: i64,
    revenue_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE orders (store text, total_cents bigint);
             INSERT INTO orders VALUES ('mitte', 4999), ('mitte', 2999), ('spandau', 999);",
        )
        .await?;
    let rows = client
        .query(
            "SELECT store, count(*) AS orders, sum(total_cents)::bigint AS revenue_cents
             FROM orders GROUP BY store ORDER BY store",
            &[],
```

```
        )
        .await?;
    let sales: Vec<StoreSales> = rows
        .iter()
        .map(|row| {
            Ok(StoreSales {
                store: row.try_get("store")?,
                orders: row.try_get("orders")?,
                revenue_cents: row.try_get("revenue_cents")?,
            })
        })
        .collect::<Result<_, tokio_postgres::Error>>()?;
    assert_eq!(
        sales[0],
        StoreSales {
            store: "mitte".into(),
            orders: 2,
            revenue_cents: 7998
        }
    );
    Ok(())
}
```

3. `avg(stars)` over an `integer` column is `numeric`, and the left join makes it NULL for a product without reviews: read an `Option<Decimal>`. If you would rather have a float, cast it, `avg(stars)::float8`, and read an `Option<f64>`.

4. All of them: `WHERE tags @> $1` ("contains"). Any of them: `WHERE tags && $1` ("overlaps"). Both take a `text[]` parameter, a `&[&str]` or `Vec<String>` on the Rust side, and both can use a GIN index on `tags`.

CHAPTER 4

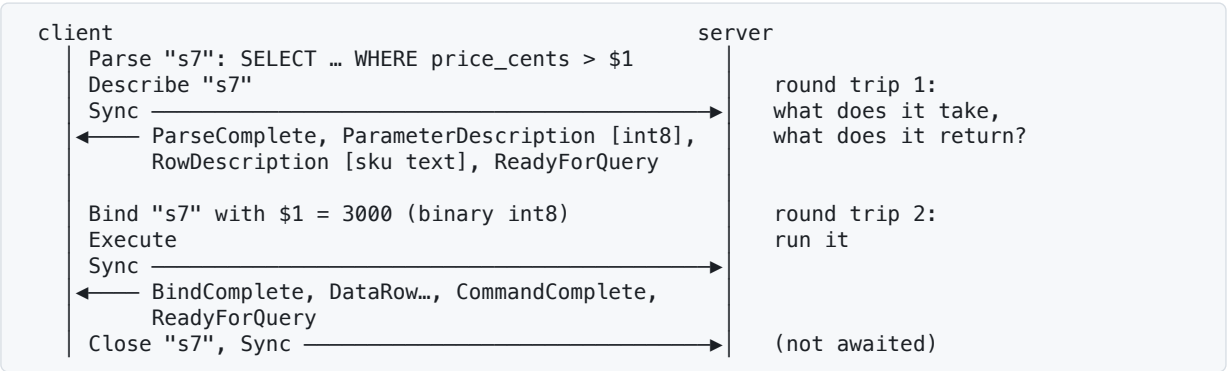
Prepared statements and the extended protocol

On localhost a query takes a few dozen microseconds, and it hardly matters how many messages it needs. Across a network, every round trip between client and server costs a fixed time (half a millisecond inside a data centre, tens of milliseconds between regions) and the count of round trips becomes the most important number in your program's performance. This chapter teaches you to count them.

It starts with what the friendly `client.query("…", &params)` actually sends, which explains why tokio-postgres's API has *statements* at all. Then come the tools for doing better: preparing a statement once and running it many times, `query_typed` for one-shot queries in a single round trip, and caching statements. Along the way you will meet the two errors that prepared statements are known for.

4.1 What `query` sends

A `Row` knows the PostgreSQL type of every column, and a parameter is checked against a PostgreSQL type before it is sent (Chapter 3). Where do those types come from? From the server, which is the only party that can tell what `$1` means in `WHERE price_cents > $1`. So when you pass SQL text to `query`, tokio-postgres does two exchanges with the server:



The first exchange *prepares* the statement: the server parses it, works out the parameter and result types, and keeps it under a name (tokio-postgres numbers them `s0`, `s1` and so on). The second *executes* it with your values, encoded in binary now that their types are known. Finally, because the statement was only needed for this one call, tokio-postgres tells the server to forget it, without waiting for an answer.

Two round trips per query is the price of passing SQL text. Most of the time you do not have to pay it.

> **Protocol lens:** these are the messages of PostgreSQL's *extended query protocol*. Chapter 2 showed the *simple* one, a single *Query* message carrying SQL text with no parameters, in one round trip. The extended protocol splits the same work into steps so that statements can be reused, values can travel separately and in binary, and results can be fetched in parts (portals, Chapter 9). Every tokio-postgres method except `simple_query` and `batch_execute` uses it.

4.2 `prepare`: one parse, many executions

`Client::prepare` does the first exchange on its own and gives you a `Statement`. Running a `Statement` costs one round trip:

```
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999), ('LAMP-01', 2999);",
        )
        .await?;

    // Parse and describe once...
    let cheaper_than = client
        .prepare("SELECT sku FROM products WHERE price_cents < $1 ORDER BY sku")
        .await?;
    // ...and the statement knows what the server worked out.
    assert_eq!(cheaper_than.params(), [Type::INT8]);
    assert_eq!(cheaper_than.columns()[0].name(), "sku");
    assert_eq!(cheaper_than.columns()[0].type_(), &Type::TEXT);

    // Run it many times: one round trip each.
    for (limit, expected) in [(9999_i64, 2), (3000, 1), (1000, 0)] {
        let rows = client.query(&cheaper_than, &[&limit]).await?;
        assert_eq!(rows.len(), expected);
    }

    // While the Statement lives, the server keeps it, in this session only.
    let prepared: Vec<String> = client
        .query_scalar("SELECT statement FROM pg_prepared_statements", &[])
        .await?;
    assert!(prepared.iter().any(|sql| sql.contains("price_cents < $1")));

    drop(cheaper_than); // sends Close: the server forgets it
    Ok(())
}
```

Every method that runs SQL accepts either text or a statement, through the `ToStatement` trait, implemented for `str`, `String` and `Statement`: `client.query("…", …)` and `client.query(&stmt, …)` are the same method. A `Statement` is cheap to clone (it is reference-counted), and the server-side statement is closed when the last clone is dropped.

A statement belongs to the session that prepared it. Use it on another connection and the server does not know the name:

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let one = playground::connect().await?;
    let other = playground::connect().await?;

    let statement = one.prepare("SELECT 1").await?;
    let error = other.query(&statement, &[]).await.unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::INVALID_SQL_STATEMENT_NAME)); // 26000
    let message = error.as_db_error().unwrap().message();
    assert!(message.starts_with("prepared statement \"s"), "{message}");
    Ok(())
}
```

That is why a pool needs a statement cache *per connection* (§4.6, Chapter 13), and why a statement prepared through a pooled connection must not be kept after the connection goes back.

> **SQL lens:** a prepared statement saves the server work too: the parse, and for repeated executions possibly the plan. PostgreSQL plans each of the first five executions with the actual values (a *custom* plan), then may switch to a *generic* plan, made once without values, if it estimates that is not worse. For skewed data, such as a `status` column where 'pending' is rare and 'done' is everywhere, a generic plan can be a bad fit for some values. The `plan_cache_mode` setting (`force_custom_plan`) overrides the choice when you find such a case.

4.3 When the server cannot infer a type

The server infers each parameter's type from its context: from the column it is compared with or assigned to, or from an operator. Usually that works. When there is no context, preparing fails:

```
use tokio_postgres::error::SqlState;
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;

    // What the server infers from context:
    let inferred = client
        .prepare("SELECT $1 + 1, $2 || 'x', $3::int8, $4")
        .await?;
    assert_eq!(
        inferred.params(),
        [Type::INT4, Type::TEXT, Type::INT8, Type::TEXT]
    );

    // No context at all: the server gives up.
    let error = client.prepare("SELECT $1 IS NULL").await.unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::INDETERMINATE_DATATYPE)); // 42P18
    assert_eq!(
        error.as_db_error().unwrap().message(),
        "could not determine data type of parameter $1"
    );

    // Fix 1: a cast in the SQL.
    let with_cast = client.prepare("SELECT $1::int8 IS NULL").await?;
    assert_eq!(with_cast.params(), [Type::INT8]);
```

```
    // Fix 2: tell prepare the types (the list may be shorter than the parameters; the
    // server infers the rest).
    let typed = client
        .prepare_typed("SELECT $1 IS NULL", &[Type::INT8])
        .await?;
    let is_null: bool = client.query_one(&typed, &[&None::<i64>]).await?.get(0);
    assert!(is_null);
    Ok(())
}
```

Note the last line of the inference examples: a bare `$4` becomes `text`. So does `SELECT $1`, which is why `client.query_one("SELECT $1", &[&42_i32])` fails on the client with "cannot convert between the Rust type `i32` and the Postgres type `text`". The server decided the type, and the Rust value has to match it, not the other way round.

A cast is usually the better fix: it documents the type where the parameter is, and it works the same in psql. `prepare_typed` is for SQL you do not control, and for the next section.

4.4 `query_typed`: one round trip, no statement

Sometimes you run a query once, or you build it dynamically and it is never the same twice. Preparing it first is a wasted round trip. `query_typed` (0.7.11+; `query_typed_one`, `query_typed_opt` and `execute_typed` since 0.7.17) skips the prepare step: you give each parameter's type along with its value, and tokio-postgres sends parse, bind and execute together, in one round trip, using an unnamed statement that nobody has to close:

```
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999), ('LAMP-01', 2999);",
        )
        .await?;

    // Each parameter comes with the PostgreSQL type it is sent as.
    let row = client
        .query_typed_one(
            "SELECT count(*) FROM products WHERE price_cents BETWEEN $1 AND $2",
            &[(&1000_i64, Type::INT8), (&3000_i64, Type::INT8)],
        )
        .await?;
    assert_eq!(row.get::<_, i64>(0), 1);

    // The types you declare must still fit the Rust values; this is checked before sending.
    let error = client
        .query_typed_one("SELECT $1 + 1", &[(&1_i64, Type::INT4)])
        .await
        .unwrap_err();
    assert_eq!(error.to_string(), "error serializing parameter 0");

    let changed = client
        .execute_typed(
            "UPDATE products SET price_cents = $1 WHERE sku = $2",
            &[(&3999_i64, Type::INT8), (&"CHAIR-01", Type::TEXT)],
        )
        .await?;
```

```
    assert_eq!(changed, 1);
    Ok(())
}
```

What you give up: the server parses and plans the statement on every call, and you are the one stating the types, so you can get them wrong. What you gain: one round trip instead of two, and no named statements on the server. That last point matters behind connection poolers that hand a different server connection to each transaction (PgBouncer in transaction mode, Chapter 13): a named statement prepared on one server connection does not exist on the next, and unnamed ones avoid the problem entirely.

Call	Round trips	Statement on the server
`query("…", params)`	2 (prepare, execute)	named, closed after the call
`prepare("…")` once, then `query(&stmt, params)`	1 + 1 per execution	named, kept while the `Statement` lives
`query_typed("…", params_with_types)`	1	unnamed, replaced by the next one
`simple_query("…")` / `batch_execute`	1	none (no parameters at all)

4.5 Counting round trips

On localhost the difference between those rows is invisible. To see it, put a slow network in the way: lagproxy, from this repository, forwards connections to PostgreSQL and holds every packet back for a few milliseconds in each direction. Start it, point `DATABASE_URL` at it, and the same program measures what each call costs:

```
cargo run -p lagproxy -- --upstream localhost:5432 --delay-ms 5    # listens on :6543
export DATABASE_URL=postgres://postgres:secret@127.0.0.1:6543/tpbook
export LAGPROXY_DELAY_MS=5
```

```
use std::time::{Duration, Instant};
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    // Through lagproxy: every round trip costs at least twice the delay.
    let client = playground::connect().await?;
    let delay: u64 = std::env::var("LAGPROXY_DELAY_MS").map_or(5, |v| v.parse().unwrap());
    let delay = Duration::from_millis(delay);
    const N: u32 = 10;

    let started = Instant::now();
    for i in 0..N as i64 {
        client.query_one("SELECT $1::int8 + 1", &[&i]).await?;
    }
    let as_text = started.elapsed() / N;

    let statement = client.prepare("SELECT $1::int8 + 1").await?;
    let started = Instant::now();
    for i in 0..N as i64 {
        client.query_one(&statement, &[&i]).await?;
    }
    let prepared = started.elapsed() / N;

    let started = Instant::now();
    for i in 0..N as i64 {
```

```
        client
            .query_typed_one("SELECT $1::int8 + 1", &[(&i, Type::INT8)])
            .await?;
    }
    let typed = started.elapsed() / N;

    println!("per query: text {as_text:?}, prepared {prepared:?}, typed {typed:?}");
    assert!(prepared >= delay * 2, "at least one round trip");
    assert!(as_text > prepared * 3 / 2, "about two round trips");
    assert!(typed < prepared * 3 / 2, "about one round trip");
    Ok(())
}
```

With a 5 ms delay each way, on the machine this book was written on:

```
per query: text 30.6ms, prepared 15.3ms, typed 15.3ms
```

A round trip through the proxy costs about 15 ms rather than 10, because a timer is never exactly on time (lagproxy's documentation explains the overshoot). The proportions are what count: passing SQL text costs two round trips, a prepared statement or `query_typed` one.

> **Pitfall:** "it's fast on my machine" is the most common way round trips hide. A request handler that runs ten small queries one after another takes 0.5 ms on localhost and 50 ms in production with a 5 ms network. Measure through lagproxy (or any latency injector) before you trust a performance number, and count the round trips of your hot paths. Chapter 7 shows how to run independent queries *without* waiting for each other.

4.6 Caching statements, and when the cache goes stale

Preparing every statement once per connection and reusing it is the way to pay one round trip per query without stating types by hand. A small cache does it: look the SQL up, prepare on a miss, keep the `Statement`.

```
use std::collections::HashMap;
use std::sync::Mutex;
use tokio_postgres::types::ToSql;
use tokio_postgres::{Client, Error, Row, Statement};

/// A client with a statement cache. One cache per connection: statements are per session.
struct CachingClient {
    client: Client,
    statements: Mutex<HashMap<String, Statement>>,
}

impl CachingClient {
    async fn prepare_cached(&self, sql: &str) -> Result<Statement, Error> {
        if let Some(statement) = self.statements.lock().unwrap().get(sql) {
            return Ok(statement.clone()); // cheap: reference-counted
        }
        // The lock is not held across the await. Two tasks may both miss and prepare the
        // same SQL; one of the statements is then dropped (and closed). Harmless.
        let statement = self.client.prepare(sql).await?;
        self.statements
            .lock()
            .unwrap()
            .insert(sql.to_owned(), statement.clone());
```

```
        Ok(statement)
    }

    async fn query(&self, sql: &str, params: &[&(dyn ToSql + Sync)]) -> Result<Vec<Row>,
Error> {
        let statement = self.prepare_cached(sql).await?;
        self.client.query(&statement, params).await
    }
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let db = CachingClient {
        client: playground::connect().await?,
        statements: Mutex::new(HashMap::new()),
    };
    for n in 0..5_i64 {
        let rows = db.query("SELECT $1::int8 * 2", &[&n]).await?;
        assert_eq!(rows[0].get::<_, i64>(0), n * 2);
    }
    assert_eq!(db.statements.lock().unwrap().len(), 1, "prepared once");
    Ok(())
}
```

You will rarely write this yourself: deadpool-postgres's pooled connections have exactly this cache built in, as `prepare_cached` (Chapter 13), and stockroom uses it everywhere. But a cache of prepared statements has a failure mode you need to know about. A statement's result columns are fixed when it is prepared. Change the table underneath, and the old statement refuses to run:

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE t (a int); INSERT INTO t VALUES (1)")
        .await?;
    let everything = client.prepare("SELECT * FROM t").await?;

    // A migration adds a column: `SELECT *` would now return two columns.
    client
        .batch_execute("ALTER TABLE t ADD COLUMN b int")
        .await?;
    let error = client.query(&everything, &[]).await.unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::FEATURE_NOT_SUPPORTED)); // 0A000
    assert_eq!(
        error.as_db_error().unwrap().message(),
        "cached plan must not change result type"
    );

    // The cure is to prepare it again.
    let everything = client.prepare("SELECT * FROM t").await?;
    assert_eq!(client.query(&everything, &[]).await?[0].len(), 2);
    Ok(())
}
```

> **Pitfall:** "cached plan must not change result type" in production means a schema change reached a service that had statements prepared against the old schema: typically a migration run by one instance while others were serving. A per-connection cache keeps handing out the broken statement until the connection is replaced. Three habits prevent it. Never `SELECT *` in application code (an added column then changes nothing). Make migrations additive. And when you must change result types, clear the statement caches or recycle the pool's connections after migrating.

Side by side – prepared statements

The task: prepare a query with one parameter once, and run it with two different values.

```
-- psql: SQL-level PREPARE / EXECUTE, which any client can send...
tpbook=# PREPARE cheaper_than(bigint) AS
tpbook-#   SELECT sku FROM products WHERE price_cents < $1 ORDER BY sku;
PREPARE
tpbook=# EXECUTE cheaper_than(3000);
   sku
---------
 LAMP-01
(1 row)

tpbook=# DEALLOCATE cheaper_than;
DEALLOCATE
-- ...or, since psql 18, the protocol-level messages tokio-postgres uses:
tpbook=# SELECT sku FROM products WHERE price_cents < $1 ORDER BY sku \parse cheaper_than
tpbook=# \bind_named cheaper_than 9999 \g
tpbook=# \close_prepared cheaper_than
```

```
// sqlx (illustration)
// Every query is prepared on first use and cached on its connection, automatically.
let skus: Vec<String> = sqlx::query_scalar("SELECT sku FROM products WHERE price_cents < $1
ORDER BY sku")
    .bind(3000_i64)
    .fetch_all(&pool)
    .await?;
// .persistent(false) opts a query out of the cache.
```

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999), ('LAMP-01', 2999);",
        )
        .await?;
    let cheaper_than = client
        .prepare("SELECT sku FROM products WHERE price_cents < $1 ORDER BY sku")
        .await?;
    let cheap: Vec<String> = client
        .query(&cheaper_than, &[&3000_i64])
        .await?
        .iter()
        .map(|row| row.get(0))
        .collect();
    let all = client.query(&cheaper_than, &[&9999_i64]).await?;
    assert_eq!((cheap, all.len()), (vec!["LAMP-01".to_owned()], 2));
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	A named, per-session statement, executed with values	Statements prepared per connection, executed with bound values	A named, per-session statement, executed with values
Different	`PREPARE` is SQL, parsed by the server like any statement; `\parse` (psql 18) uses the protocol	Caching is automatic and invisible: you never hold a statement	Explicit: you prepare and hold a `Statement` (or use deadpool's `prepare_cached`); text queries pay a second round trip; `query_typed` avoids both

In stockroom

Every repository function in stockroom prepares through the pool's cache, so each statement is parsed once per connection and every later call costs one round trip. The catalog's paginated listing:

```
// stockroom/src/repo/products.rs (excerpt)
pub async fn page(
    db: &impl GenericClient,
    after: Option<&Sku>,
    tag: Option<&str>,
    limit: i64,
) -> Result<Vec<Product>, AppError> {
    let stmt = db
        .prepare_cached(
            "SELECT id, sku, name, price_cents, tags, attributes, version, created_at,
updated_at
             FROM products
             WHERE sku > coalesce($1, '') AND ($2::text IS NULL OR tags @> ARRAY[$2])
             ORDER BY sku
             LIMIT $3",
        )
        .await?;
    from_rows(&db.query(&stmt, &[&after, &tag, &limit]).await?)
}
```

Look at the parameters through this chapter's eyes. `$3` gets its type, `bigint`, from `LIMIT`. `$1` is compared with a `sku` through `coalesce($1, '')`, which makes it `text`. And `$2` appears twice, so it carries an explicit cast at its first use, `$2::text`, because `$2 IS NULL` alone would be the "could not determine data type" error from §4.3. The column list is spelled out rather than `*`, so a migration that adds a column to `products` cannot break the cached statement.

stockroom also guards against the stale-statement problem at startup: `serve` runs the migrations *before* it creates the pool (app.rs), so no connection has prepared anything against the old schema.

The tiny psql in this repository uses `prepare` for something other than speed: its `--describe` mode prepares the statement and prints what the server worked out, without ever running it:

```
// apps/pgq/src/lib.rs (excerpt)
pub async fn describe(client: &Client, sql: &str) -> Result<String, Error> {
    let statement = client.prepare(sql).await?;
    // … renders statement.params() and statement.columns() as two tables …
}
```

```
$ cargo run -p pgq -- --describe -c 'SELECT relname, relpages FROM pg_class WHERE oid = $1'
 parameter | type
-----------+------
 $1        | oid

 column   | type | from table
----------+------+--------------------
 relname  | name | oid 1259, column 2
 relpages | int4 | oid 1259, column 10
```

Summary

- Passing SQL text to `query`, `execute` and friends costs two round trips: prepare (types in, types out) and execute. A temporary named statement is created and closed.
- `prepare` returns a `Statement` with `params()` and `columns()`. Executing it costs one round trip. Statements belong to one session: using one elsewhere fails with 26000.
- When a parameter's type cannot be inferred (42P18), add a cast (`$1::int8`) or use `prepare_typed`. The server decides parameter types; Rust values must fit them.
- `query_typed`, `query_typed_one`, `query_typed_opt` and `execute_typed` take the types with the values and need one round trip, with no named statement: good for one-off and dynamic SQL, and behind transaction-mode poolers.
- Cache statements per connection (deadpool's `prepare_cached` does). After a schema change, cached statements can fail with "cached plan must not change result type": avoid `SELECT *`, migrate additively, and re-prepare after migrating.
- Measure through added latency (lagproxy): round trips only show on a real network.

Exercises

1. **Count them yourself.** With lagproxy at a 10 ms delay, time a loop of 20 `execute` calls that each insert one row into a temporary table, first with SQL text and then with a prepared statement. Predict both times before you run it. *Hint:* at a 10 ms delay, a round trip through lagproxy measured about 26 ms.
2. **Infer it.** For each of these, predict the type the server gives `$1`, then check with `prepare` or `pgq --describe`: `SELECT $1 = 1`, `SELECT length($1)`, `SELECT now() - $1`, `SELECT ARRAY[$1, 2]`, `SELECT coalesce($1, 0.5)`.
3. **A dynamic query in one round trip.** Rewrite Chapter 2's `search` (optional filters) with `query_typed`, so that every combination of filters costs one round trip. *Hint:* the parameter list becomes `Vec<(&(dyn ToSql + Sync), Type)>`.
4. **Stale.** Using the cache from §4.6, write a test that shows the stale-statement failure after `ALTER TABLE`, then make `CachingClient::query` recover by itself: on SQLSTATE 0A000, drop the cached statement, prepare it again and retry once.

Solutions

1. About $2 \times 20 \times 26$ ms ≈ 1.04 s with text, and 20×26 ms ≈ 0.52 s (plus one round trip for `prepare`) with the statement. Every `execute` waits for its answer before the next one is sent, so the loop is a chain of round trips. Chapter 7 breaks the chain.

2. `$1 = 1` makes it `integer` (from the other operand), and `length($1)` makes it `text` (the most common `length` takes text). `now() - $1` makes it **`timestamptz`**, not `interval`: of the `-` operators with a `timestamptz` on the left, the server prefers the one whose right side has the same type, so `now() - $1` subtracts two timestamps and returns an interval. If you meant "now minus a duration", write `now() - $1::interval`. `ARRAY[$1, 2]` gives `integer`, and `coalesce($1, 0.5)` gives `numeric`, the type of the other argument. When the answer surprises you, cast.

4.

```
use std::collections::HashMap;
use std::sync::Mutex;
use tokio_postgres::error::SqlState;
use tokio_postgres::types::ToSql;
use tokio_postgres::{Client, Error, Row, Statement};

struct CachingClient {
    client: Client,
    statements: Mutex<HashMap<String, Statement>>,
}

impl CachingClient {
    async fn prepare_cached(&self, sql: &str) -> Result<Statement, Error> {
        if let Some(statement) = self.statements.lock().unwrap().get(sql) {
            return Ok(statement.clone());
        }
        let statement = self.client.prepare(sql).await?;
        self.statements
            .lock()
            .unwrap()
            .insert(sql.to_owned(), statement.clone());
        Ok(statement)
    }

    async fn query(&self, sql: &str, params: &[&(dyn ToSql + Sync)]) -> Result<Vec<Row>,
Error> {
        let statement = self.prepare_cached(sql).await?;
        match self.client.query(&statement, params).await {
            Err(e) if e.code() == Some(&SqlState::FEATURE_NOT_SUPPORTED) => {
                // Stale after a schema change: forget it, prepare again, retry once.
                self.statements.lock().unwrap().remove(sql);
                let statement = self.prepare_cached(sql).await?;
                self.client.query(&statement, params).await
            }
            other => other,
        }
    }
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let db = CachingClient {
        client: playground::connect().await?,
        statements: Mutex::new(HashMap::new()),
    };
    db.client
        .batch_execute("CREATE TEMP TABLE t (a int); INSERT INTO t VALUES (1)")
        .await?;
    assert_eq!(db.query("SELECT * FROM t", &[]).await?[0].len(), 1);
    db.client
```

```
        .batch_execute("ALTER TABLE t ADD COLUMN b int")
        .await?;
    assert_eq!(
        db.query("SELECT * FROM t", &[]).await?[0].len(),
        2,
        "recovered"
    );
    Ok(())
}
```

0A000 is "feature not supported", a broad code. Matching on it alone is acceptable here, because the retry is harmless; a stricter version also checks the message.

CHAPTER 5

Custom types: ToSql and FromSql

Chapter 3's table covers the types both languages have in common. But a well-designed schema has types of its own: an `order_status` enum, an `address` composite, a `sku` domain that only accepts valid codes. A well-designed Rust program has its own too: a `Cents` newtype that cannot be confused with a quantity, an `Email` that is known to be valid. This chapter connects the two.

There are two ways. You can implement the two conversion traits, `ToSql` and `FromSql`, by hand. That takes a dozen lines, and it is the way to add validation or to support a type nobody else supports. Or you can derive them for the common shapes: newtypes, enums, domains and composites. You will meet both in stockroom, and you will also meet the two things that make custom types tricky in practice: how domain types are described on the wire, and what happens when an enum changes underneath a running program.

5.1 The two traits

`ToSql` turns a Rust value into the bytes of a PostgreSQL value, for a given PostgreSQL type; `FromSql` does the reverse. Both live in the `postgres-types` crate, re-exported as `tokio_postgres::types`. Stripped of details:

```
pub trait ToSql: fmt::Debug {
    /// Write `self` in the binary format of `ty` to `out`.
    fn to_sql(&self, ty: &Type, out: &mut BytesMut) -> Result<IsNull, Box<dyn Error + Sync +
Send>>;
    /// Can this Rust type be sent as a value of `ty`?
    fn accepts(ty: &Type) -> bool;
    /// `to_sql`, after checking `accepts`; written for you by the `to_sql_checked!()`
macro.
    fn to_sql_checked(&self, ty: &Type, out: &mut BytesMut) -> Result<IsNull, Box<dyn Error
+ Sync + Send>>;
}

pub trait FromSql<'a>: Sized {
    /// Read a value of `ty` from its binary format in `raw`.
    fn from_sql(ty: &Type, raw: &'a [u8]) -> Result<Self, Box<dyn Error + Sync + Send>>;
    /// Can a value of `ty` be read as this Rust type?
    fn accepts(ty: &Type) -> bool;
    /// What NULL means. The default is an error (`WasNull`); `Option<T>` overrides it.
    fn from_sql_null(ty: &Type) -> Result<Self, Box<dyn Error + Sync + Send>> { … }
}
```

Three things are worth noting before you implement them.

- **`accepts` is the type check** you met in Chapter 3. tokio-postgres calls it with the type the server described, before encoding a parameter or decoding a column, and turns a `false` into the "cannot convert between the Rust type … and the Postgres type …" error.
- **Everything is binary.** tokio-postgres asks for binary values, so `raw` is the type's binary representation (big-endian integers, UTF-8 text, and so on), not text. You rarely write it by hand: delegate to the implementation of a type that already knows the format.
- **Errors are boxed.** Any error type converts into `Box<dyn Error + Sync + Send>`, including a `String`. tokio-postgres wraps yours as the cause of its own error.

5.2 A newtype by hand

The most common custom type is a validated newtype: an `Email` is a `String` that has been checked. It travels as text, so the implementation delegates encoding and decoding to `&str` and adds the check on the way out of the database as well as on the way in:

```
use bytes::BytesMut;
use postgres_types::{FromSql, IsNull, ToSql, Type, to_sql_checked};
use std::error::Error;

/// An email address, known to be well-formed.
#[derive(Debug, Clone, PartialEq)]
struct Email(String);

impl Email {
    fn parse(text: &str) -> Result<Email, String> {
        match text.split_once('@') {
            Some((user, host)) if !user.is_empty() && host.contains('.') => {
                Ok(Email(text.to_owned()))
            }
            _ => Err(format!("{text:?} is not an email address")),
        }
    }
}

impl ToSql for Email {
    fn to_sql(
        &self,
        ty: &Type,
        out: &mut BytesMut,
    ) -> Result<IsNull, Box<dyn Error + Sync + Send>> {
        self.0.as_str().to_sql(ty, out) // text's binary format: the UTF-8 bytes
    }

    fn accepts(ty: &Type) -> bool {
        <&str as ToSql>::accepts(ty) // text, varchar, …
    }

    to_sql_checked!();
}

impl<'a> FromSql<'a> for Email {
    fn from_sql(ty: &Type, raw: &'a [u8]) -> Result<Self, Box<dyn Error + Sync + Send>> {
        let text = <&str as FromSql>::from_sql(ty, raw)?;
        Ok(Email::parse(text)?) // a bad value in the database is an error, not an Email
    }

    fn accepts(ty: &Type) -> bool {
        <&str as FromSql>::accepts(ty)
    }
}
```

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE customers (id int, email text)")
        .await?;

    let ada = Email::parse("ada@example.com").expect("valid");
    client
        .execute("INSERT INTO customers VALUES (1, $1)", &[&ada])
        .await?;
    let back: Email = client
        .query_one("SELECT email FROM customers WHERE id = 1", &[])
        .await?
        .get(0);
    assert_eq!(back, ada);

    // Something not written by this program: the check runs on the way out too.
    client
        .execute("INSERT INTO customers VALUES (2, 'not an address')", &[])
        .await?;
    let row = client
        .query_one("SELECT email FROM customers WHERE id = 2", &[])
        .await?;
    let error = row.try_get::<_, Email>("email").unwrap_err();
    let cause = Error::source(&error).expect("our message").to_string();
    assert_eq!(cause, "\"not an address\" is not an email address");

    // accepts() keeps it to text-like types.
    let refused = client
        .execute("SELECT $1::int4", &[&ada])
        .await
        .unwrap_err();
    assert_eq!(refused.to_string(), "error serializing parameter 0");
    Ok(())
}
```

`to_sql_checked!()` writes the third method of `ToSql` for you: it calls `accepts`, returns the "cannot convert" error if it says no, and otherwise calls `to_sql`. Every hand-written `ToSql` ends with it.

> **Rust refresher:** `<&str as ToSql>::accepts(ty)` is the fully qualified way to call a trait function on a specific type: "`accepts`, as `&str` implements it in `ToSql`". You need it because `&str` implements both `ToSql::accepts` and `FromSql::accepts`, and the compiler cannot guess which one you mean from `<&str>::accepts`.

5.3 Deriving: `transparent` newtypes

Most newtypes need no validation at all: they exist so that the compiler can tell a price from a quantity. For those, derive the traits and say that the type is *transparent*, meaning that it travels exactly like its single field:

```
use postgres_types::{FromSql, ToSql};

/// Money, as a whole number of cents.
#[derive(Debug, Clone, Copy, PartialEq, ToSql, FromSql)]
#[postgres(transparent)]
struct Cents(i64);

#[tokio::main]
```

```
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE prices (sku text, price_cents bigint)")
        .await?;
    client
        .execute(
            "INSERT INTO prices VALUES ('CHAIR-01', $1)",
            &[&Cents(4999)],
        )
        .await?;

    let price: Cents = client
        .query_one("SELECT price_cents FROM prices", &[])
        .await?
        .get(0);
    assert_eq!(price, Cents(4999));

    // Transparent all the way down: arrays of Cents are bigint[].
    let doubled: Vec<Cents> = client
        .query_one(
            "SELECT array_agg(x * 2) FROM unnest($1::bigint[]) AS x",
            &[&vec![Cents(1), Cents(2)]],
        )
        .await?
        .get(0);
    assert_eq!(doubled, [Cents(2), Cents(4)]);
    Ok(())
}
```

`Cents` accepts exactly what `i64` accepts, `bigint`, and nothing else: sending it into an `integer` column is still a type error, which is the point of the newtype.

5.4 Enums

A PostgreSQL enum is a type with a fixed list of labels. A Rust enum of unit variants maps onto it with the derive. The type's name and the labels must match; `rename_all` saves you writing each label when they follow a convention:

```
use postgres_types::{FromSql, ToSql};

#[derive(Debug, Clone, Copy, PartialEq, ToSql, FromSql)]
#[postgres(name = "order_status", rename_all = "lowercase")]
enum OrderStatus {
    Pending,
    Paid,
    Cancelled,
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    // A temporary type, like the temporary tables: it disappears with the session.
    client
        .batch_execute(
            "CREATE TYPE pg_temp.order_status AS ENUM ('pending', 'paid', 'cancelled');
             CREATE TEMP TABLE orders (id int, status order_status);
             INSERT INTO orders VALUES (1, 'pending'), (2, 'paid'), (3, 'cancelled');",
        )
        .await?;

    client
        .execute(
            "UPDATE orders SET status = $1 WHERE id = 1",
```

```
            &[&OrderStatus::Paid],
        )
        .await?;

    // An array of enum values for = ANY, and enum values back.
    let open = vec![OrderStatus::Pending, OrderStatus::Paid];
    let rows = client
        .query(
            "SELECT id, status FROM orders WHERE status = ANY($1) ORDER BY id",
            &[&open],
        )
        .await?;
    let found: Vec<(i32, OrderStatus)> = rows
        .iter()
        .map(|row| (row.get("id"), row.get("status")))
        .collect();
    assert_eq!(found, [(1, OrderStatus::Paid), (2, OrderStatus::Paid)]);
    Ok(())
}
```

The derive's `accepts` checks the type's *name* (`order_status`, in any schema) and its *labels*, which tokio-postgres looked up from the server the first time it met the type. By default the labels must match the variants exactly: same set, nothing extra on either side. That strictness catches a typo in a `rename` on the first query. It also has a consequence that bites in production:

```
use postgres_types::{FromSql, ToSql};

#[derive(Debug, Clone, Copy, PartialEq, ToSql, FromSql)]
#[postgres(name = "order_status", rename_all = "lowercase")]
enum Strict {
    Pending,
    Paid,
}

#[derive(Debug, Clone, Copy, PartialEq, ToSql, FromSql)]
#[postgres(name = "order_status", rename_all = "lowercase", allow_mismatch)]
enum Lenient {
    Pending,
    Paid,
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TYPE pg_temp.order_status AS ENUM ('pending', 'paid')")
        .await?;
    let paid = "SELECT 'paid'::order_status";
    assert_eq!(
        client.query_one(paid, &[]).await?.get::<_, Strict>(0),
        Strict::Paid
    );

    // A migration adds a label the program does not know yet.
    client
        .batch_execute("ALTER TYPE order_status ADD VALUE 'refunded'")
        .await?;
    // A connection that loads the type afresh (any new connection, or this one after
    // clearing its type cache) now sees three labels where the strict enum has two:
    client.clear_type_cache();
    let row = client.query_one(paid, &[]).await?;
    assert!(
        row.try_get::<_, Strict>(0).is_err(),
        "even 'paid' is refused"
    );

    // allow_mismatch only fails on a label the enum lacks, when a row actually holds it.
```

```
    assert_eq!(row.get::<_, Lenient>(0), Lenient::Paid);
    let refunded = client
        .query_one("SELECT 'refunded'::order_status", &[])
        .await?;
    let error = refunded.try_get::<_, Lenient>(0).unwrap_err();
    let cause = std::error::Error::source(&error).unwrap().to_string();
    assert_eq!(cause, "invalid variant `refunded`");
    Ok(())
}
```

> **Pitfall:** with a strict enum, `ALTER TYPE … ADD VALUE` in a migration breaks every running instance of your service, gradually: each new pooled connection loads the new label list and then refuses *every* value of the type, not just the new one. The code and the database would have to change at the same instant, which never happens. For enums that can grow, add `allow_mismatch` and deploy in either order: the database first (the old code fails only on rows with the new label), or the code first (the new variant is simply never sent).

5.5 Domains

A domain is a base type with a constraint: `CREATE DOMAIN sku AS text CHECK (…)`. You would expect it to behave exactly like its base type, and on the way *out* it does: a column of type `sku` is described as `text`. On the way *in*, it does not. A parameter that goes into a `sku` column is described as the domain itself, and plain Rust types refuse it:

```
use postgres_types::{FromSql, ToSql};
use tokio_postgres::types::{Kind, Type};

/// Derived for a domain: a one-field tuple struct with the domain's name.
#[derive(Debug, Clone, PartialEq, ToSql, FromSql)]
#[postgres(name = "sku")]
struct Sku(String);

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE DOMAIN pg_temp.sku AS text CHECK (VALUE ~ '^[A-Z0-9]{3,12}(-[A-Z0-9]
{1,6})?$');
             CREATE TEMP TABLE products (sku sku PRIMARY KEY, name text);",
        )
        .await?;

    let insert = client
        .prepare("INSERT INTO products VALUES ($1, $2)")
        .await?;
    // $1 targets the sku column: it is described as the domain, not as text.
    assert_eq!(insert.params()[0].name(), "sku");
    assert!(matches!(insert.params()[0].kind(), Kind::Domain(base) if *base == Type::TEXT));

    // So a plain &str is refused, by the client, before anything is sent...
    let refused = client
        .execute(&insert, &[&"CHAIR-01", &"Chair"])
        .await
        .unwrap_err();
    assert!(refused.as_db_error().is_none());
```

```
    // ...while the derived domain type is accepted, and the server checks the constraint.
    client
        .execute(&insert, &[&Sku("CHAIR-01".into()), &"Chair"])
        .await?;
    let bad = client
        .execute(&insert, &[&Sku("chair".into()), &"Chair"])
        .await
        .unwrap_err();
    assert_eq!(bad.code().map(|c| c.code()), Some("23514")); // check_violation

    // A cast makes the parameter plain text again, if you prefer &str.
    client
        .execute(
            "INSERT INTO products VALUES ($1::text, $2)",
            &[&"LAMP-01", &"Lamp"],
        )
        .await?;

    // On the way out, the column is described as text, and both types read it.
    let row = client
        .query_one("SELECT sku FROM products WHERE name = 'Lamp'", &[])
        .await?;
    assert_eq!(row.columns()[0].type_(), &Type::TEXT);
    assert_eq!(row.get::<_, Sku>(0), Sku("LAMP-01".into()));
    assert_eq!(row.get::<_, &str>(0), "LAMP-01");
    Ok(())
}
```

So for each domain you choose one of three approaches:

Approach	Parameters	Results	When
A derived domain new-type, `#[postgres(name = "…")] struct T(Base)`	accepted as the domain	read from the base type too	the domain has a Rust type of its own
A hand-written `accepts` that takes the domain *and* the base type	accepted either way	accepted either way	you also want validation in Rust (stockroom's `Sku`)
A cast in the SQL, `$1::text`	the base type	the base type	occasional use, or SQL you do not control

Comparisons are not affected: in `WHERE sku = $1` the server types `$1` as the base type, so a `&str` works there.

> **SQL lens:** domains are a cheap way to give a constraint a name and reuse it across tables, and the server enforces it everywhere the domain is used. One limit is new in PostgreSQL 18: a *virtual* generated column (`GENERATED ALWAYS AS (…) VIRTUAL`) cannot use a user-defined type, and a domain counts, so a virtual column cannot be computed from domain-typed columns. stockroom's `stock.available` ran into exactly that.

5.6 Composite types

A composite type is a named record: `CREATE TYPE address AS (street text, city text, …)`. Tables can have composite columns, and a query can produce a composite value with `ROW(…)`. A Rust struct with named fields maps onto it, field by field, by name:

```
use postgres_types::{FromSql, ToSql};

#[derive(Debug, Clone, PartialEq, ToSql, FromSql)]
#[postgres(name = "address")]
struct Address {
    street: String,
    city: String,
    postcode: Option<String>, // a field that may be NULL
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TYPE pg_temp.address AS (street text, city text, postcode text);
             CREATE TEMP TABLE stores (code text, address address);",
        )
        .await?;

    let mitte = Address {
        street: "Main 1".into(),
        city: "Berlin".into(),
        postcode: Some("10115".into()),
    };
    client
        .execute("INSERT INTO stores VALUES ('mitte', $1)", &[&mitte])
        .await?;

    // The whole value, or one field of it, in SQL.
    let row = client
        .query_one(
            "SELECT address, (address).city AS city FROM stores WHERE code = 'mitte'",
            &[],
        )
        .await?;
    assert_eq!(row.get::<_, Address>("address"), mitte);
    assert_eq!(row.get::<_, &str>("city"), "Berlin");

    // Arrays of composites work like any array.
    let more = vec![
        Address {
            street: "Ring 2".into(),
            city: "Potsdam".into(),
            postcode: None,
        },
        Address {
            street: "Allee 3".into(),
            city: "Leipzig".into(),
            postcode: Some("04109".into()),
        },
    ];
    let codes = vec!["ring", "allee"];
    // unnest in the select list keeps each address whole...
    client
        .execute(
            "INSERT INTO stores SELECT unnest($1::text[]), unnest($2::address[])",
            &[&codes, &more],
        )
        .await?;
    // ...while in FROM, a function returning a composite is expanded into its fields.
    let error = client
```

```
        .execute(
            "INSERT INTO stores SELECT * FROM unnest($1::text[], $2::address[])",
            &[&codes, &more],
        )
        .await
        .unwrap_err();
    assert_eq!(
        error.as_db_error().unwrap().message(),
        "INSERT has more expressions than target columns"
    );

    let count: i64 = client
        .query_one("SELECT count(*) FROM stores", &[])
        .await?
        .get(0);
    assert_eq!(count, 3);
    Ok(())
}
```

The derive matches fields by name, so their order in the struct does not matter, and it checks that the type has exactly these fields. A field that can be NULL needs an `Option`, just like a column. `#[postgres(name = "…")]` on a field renames it, and `rename_all` works here too.

5.7 Behind the scenes: `Type`, `Kind` and the type cache

Built-in types are constants: `Type::INT8`, `Type::TEXT`. A type the server defines at run time (an enum, a domain, a composite, or an extension's type) has an OID that tokio-postgres cannot know in advance. The first time it meets one in a statement's description, it looks the type up in the server's catalog, with a few extra queries, and builds a `Type` whose `kind()` describes it:

`Kind`	What it is
`Simple`	A base type, such as `int8` or `text`
`Enum(labels)`	An enum, with its labels
`Domain(base)`	A domain over `base`
`Composite(fields)`	A composite type, with each field's name and type
`Array(member)`	An array of `member`
`Range(member)`, `Multirange(member)`	A range type
`Pseudo`	A pseudo-type such as `void` or `record`

The derives' `accepts` read exactly this: the enum's labels, the domain's base type, the composite's fields.

These lookups are cached *per connection*, for the connection's lifetime. That is why the strict enum in §5.4 kept working until the cache was cleared: the connection still held the old label list. Two consequences follow:

- After `ALTER TYPE` (new enum labels, changed composite fields), connections that existed before keep the old description until they are closed or call `Client::clear_type_cache()`. stockroom's migration runner calls it after applying migrations, for its own connection.
- The first query on a fresh connection that involves a custom type pays for the lookup: a few extra round trips, once. A pool hides this cost behind connection reuse (Chapter 13).

Side by side — a PostgreSQL enum

The task: define an `order_status` enum, store a value, and read it back as a typed value.

```
-- psql
tpbook=# CREATE TYPE order_status AS ENUM ('pending', 'paid', 'cancelled');
tpbook=# \dT+ order_status
                                       List of data types
 Schema |     Name     | Internal name | Size | Elements  |  Owner   | Access privileges |
Description
--------+--------------+---------------+------+-----------+----------+-------------------
+-------------
 public | order_status | order_status  | 4    | pending  +| postgres |                   |
        |              |               |      | paid     +|          |                   |
        |              |               |      | cancelled |          |                   |
(1 row)

tpbook=# SELECT 'paid'::order_status < 'cancelled';   -- labels are ordered as declared
 ?column?
----------
 t
(1 row)
```

```
// sqlx (illustration)
#[derive(sqlx::Type)]
#[sqlx(type_name = "order_status", rename_all = "lowercase")]
enum OrderStatus { Pending, Paid, Cancelled }

let status: OrderStatus = sqlx::query_scalar("SELECT
'paid'::order_status").fetch_one(&pool).await?;
```

```
use postgres_types::{FromSql, ToSql};

#[derive(Debug, Clone, Copy, PartialEq, ToSql, FromSql)]
#[postgres(name = "order_status", rename_all = "lowercase")]
enum OrderStatus {
    Pending,
    Paid,
    Cancelled,
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TYPE pg_temp.order_status AS ENUM ('pending', 'paid',
'cancelled')")
        .await?;
    let status: OrderStatus = client
        .query_one("SELECT $1::order_status", &[&OrderStatus::Paid])
        .await?
        .get(0);
    assert_eq!(status, OrderStatus::Paid);
    let pending = OrderStatus::Pending;
    assert_ne!(status, pending);
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	An enum is a type with ordered labels	A derive maps the type name and the labels	A derive maps the type name and the labels
Different	Labels are text you type	Matches labels by name when encoding and decoding	By default, the label *set* must match exactly, checked against

psql	sqlx	tokio-postgres
		the labels looked up from the server (per connection); `allow_mismatch` relaxes that to per-value

In stockroom

stockroom's domain/ module has one of each kind. `Cents` is a transparent newtype:

```
// stockroom/src/domain/cents.rs (excerpt)
#[serde(transparent)]
#[postgres(transparent)]
pub struct Cents(pub i64);
```

`OrderStatus` is the enum, with `allow_mismatch` and the reason written next to it:

```
// stockroom/src/domain/order_status.rs (excerpt)
/// `allow_mismatch` matters in production. Without it, the derive accepts the database type
/// only if its labels are *exactly* these five: a migration that adds a sixth status would
/// make every new connection refuse every `order_status` value, `'paid'` included, until
/// this enum gets the variant too. With it, only an unknown label fails, and only when a
row
/// actually holds it, so the database can be migrated ahead of the code.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize, ToSql, FromSql)]
#[serde(rename_all = "lowercase")]
#[postgres(name = "order_status", rename_all = "lowercase", allow_mismatch)]
pub enum OrderStatus {
    Pending,
    Paid,
    Ready,
    Collected,
    Cancelled,
}
```

A test in tests/types.rs pins that behaviour down: it adds a label to the database and checks that fresh connections can still read the others.

`Quantity` is a derived domain newtype for `positive_quantity` (which order lines use), with its own validation on the Rust side through serde. `Address` is the composite. `Sku` is written by hand, because it wants both a domain-aware `accepts` and validation when reading:

```
// stockroom/src/domain/sku.rs (excerpt)
/// `text`, `varchar` and friends, or the `sku` domain over one of them.
fn accepts_sku(ty: &Type) -> bool {
    match ty.kind() {
        Kind::Domain(base) => ty.name() == "sku" && <&str as ToSql>::accepts(base),
        _ => <&str as ToSql>::accepts(ty),
    }
}

impl ToSql for Sku {
    fn to_sql(
        &self,
        _ty: &Type,
        out: &mut BytesMut,
    ) -> Result<IsNull, Box<dyn Error + Sync + Send>> {
        // The binary format of `text` (and so of a domain over it) is just the UTF-8 bytes.
```

```
        out.extend_from_slice(self.0.as_bytes());
        Ok(IsNull::No)
    }

    fn accepts(ty: &Type) -> bool {
        accepts_sku(ty)
    }

    to_sql_checked!();
}
```

Instead of delegating, `to_sql` writes the bytes itself, which shows how simple text's binary format is. The seed command inserts stores as three arrays zipped by `unnest`, one of them an array of the composite, and its comment records §5.6's lesson:

```
// stockroom/src/cli/seed.rs (excerpt)
let store_ids: Vec<Uuid> = tx
    .query(
        "INSERT INTO stores (code, name, address)
         SELECT unnest($1::text[]), unnest($2::text[]), unnest($3::address[])
         RETURNING id",
        &[&codes, &names, &addresses],
    )
    .await?
```

Summary

- `ToSql` and `FromSql` encode and decode the binary format of a PostgreSQL type; `accepts` is the type check behind "cannot convert". End a hand-written `ToSql` with `to_sql_checked!()`.
- Hand-written implementations usually delegate to an existing type and add validation. A `FromSql` that validates turns bad data into a conversion error.
- Derives cover newtypes (`transparent`), enums (`name`, `rename_all`, `allow_mismatch`), domains (a one-field tuple struct with the domain's `name`) and composites (named fields).
- A derived enum checks the whole label set by default; for enums that grow, use `allow_mismatch`, or every new connection refuses every value after a migration.
- A domain-typed parameter is described as the domain, so plain base types are refused; result columns are described as the base type.
- In `FROM`, `unnest` of a composite array expands the fields; in the select list it does not.
- Custom types are looked up once per connection and cached; `clear_type_cache` forgets them.

Exercises

1. **A percentage.** Write `struct Percent(u8)` that maps to `smallint`, accepts 0 to 100 on both sides, and refuses anything else with a clear message. *Hint:* delegate to `i16`'s implementations, and convert with `i16::from` and `u8::try_from`.
2. **Evolve an enum.** With a strict enum and a real (not temporary) type, open two connections, add a label with one of them, and check what each connection can read. Explain the difference.
3. **A composite with a nested composite.** Make a `geo` composite `(lat float8, lon float8)` and an `address` composite that has a `location geo` field. Derive both in Rust, and round trip a value.

4. **`interval`, by hand.** tokio-postgres has no mapping for `interval`. Its binary format is 16 bytes: a big-endian `i64` of microseconds, then an `i32` of days, then an `i32` of months. Write `struct PgInterval { months: i32, days: i32, microseconds: i64 }` with a `FromSql`, and read `interval '1 year 2 months 3 days 04:05:06.5'`. *Hint:* `postgres_types::accepts!(INTERVAL)` writes `accepts` for you.

Solutions

2. The connection that ran `ALTER TYPE` still holds the type's old description in its cache, so its strict enum keeps working (until it calls `clear_type_cache`). The second connection behaves the same *if* it had already used the type. A third connection opened afterwards looks the type up fresh, sees one label too many, and refuses every value. That is how a migration breaks a service gradually, as its pool replaces connections.

4.

```
use postgres_types::{FromSql, Type, accepts};
use std::error::Error;

#[derive(Debug, PartialEq)]
struct PgInterval {
    months: i32,
    days: i32,
    microseconds: i64,
}

impl<'a> FromSql<'a> for PgInterval {
    fn from_sql(_: &Type, raw: &'a [u8]) -> Result<Self, Box<dyn Error + Sync + Send>> {
        let raw: &[u8; 16] = raw.try_into()?; // anything else is an error, not a panic
        Ok(PgInterval {
            microseconds: i64::from_be_bytes(raw[0..8].try_into()?),
            days: i32::from_be_bytes(raw[8..12].try_into()?),
            months: i32::from_be_bytes(raw[12..16].try_into()?),
        })
    }

    accepts!(INTERVAL);
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let interval: PgInterval = client
        .query_one("SELECT interval '1 year 2 months 3 days 04:05:06.5'", &[])
        .await?
        .get(0);
    assert_eq!(
        interval,
        PgInterval {
            months: 14,
            days: 3,
            microseconds: ((4 * 60 + 5) * 60 + 6) * 1_000_000 + 500_000,
        }
    );
    Ok(())
}
```

The three parts stay separate because they are not interchangeable. A month is not a fixed number of days, and a day is not always 24 hours across a daylight-saving change, so PostgreSQL keeps them apart and so should you. Converting to a single `Duration` is only correct when `months` and `days` are zero.

CHAPTER 6

Errors

Every chapter so far has produced errors: a refused connection, a wrong type, a missing row, a violated constraint. They all arrive as the same Rust type, `tokio_postgres::Error`, and the first thing to learn is that its `Display` text almost never tells you what happened. The information is there, one level down, in a structure the server filled in with remarkable care: a five-character SQLSTATE code, the constraint and table involved, a detail, a hint, the position of a syntax error.

This chapter takes the error apart: its causes, the server's `DbError` and its fields, and the `SqlState` codes worth matching on. Then it builds what an application needs on top: one error type for your program, classified once, with messages a user can be shown and details that go only to the log. It ends with two things that are not errors in the usual sense: a transaction that refuses to continue after one, and notices.

6.1 One type, many causes

`tokio_postgres::Error` covers everything that can go wrong, from a refused TCP connection to a deadlock detected by the server. Internally it records a *kind* and an optional *cause*. The kind is private, so what you can ask an error is limited to four questions:

Method	Answers
`as_db_error()`	Did the server report this? Then here is its `DbError`
`code()`	The SQLSTATE, if the server reported it (a shortcut through `as_db_error`)
`is_closed()`	Is it because the connection is closed?
`source()` (from `std::error::Error`)	The underlying cause: the `DbError`, an `io::Error`, a conversion error

Here is what each kind of failure looks like from the outside, as observed on PostgreSQL 18:

What happened	`Display`	`code()`	`source()`
A constraint, a syntax error, a deadlock: anything the server reports	`db error`	the SQLSTATE	the server's message, formatted as `ERROR: … DETAIL: …`
The connection was refused	`error connecting to server`	—	`Connection refused (os error 61)`
The connection is closed	`connection closed`	—	—

What happened	`Display`	`code()`	`source()`
A value did not fit the column's type (reading)	`error deserializing column 0`	—	`cannot convert between the Rust type … and the Postgres type …`
A value did not fit the parameter's type (sending)	`error serializing parameter 0`	—	the same "cannot convert"
No such column	`invalid column `x``	—	—
Wrong number of parameters	`expected 1 parameters but got 0`	—	—
`query_one`/`query_opt` saw the wrong number of rows	`query returned an unexpected number of rows`	—	—
A connection string that does not parse	`invalid connection string`	—	`invalid value for option `port``

So `Display` is a category, and the explanation is the cause. To see both, walk the chain:

```
use std::error::Error;

/// An error and every cause under it.
fn report(error: &dyn Error) -> String {
    let mut text = error.to_string();
    let mut cause = error.source();
    while let Some(c) = cause {
        text.push_str(&format!("\n  caused by: {c}"));
        cause = c.source();
    }
    text
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999);",
        )
        .await?;

    // From the server: the DbError is the cause.
    let duplicate = client
        .execute("INSERT INTO products VALUES ('CHAIR-01', 1)", &[])
        .await
        .unwrap_err();
    let text = report(&duplicate);
    assert!(text.starts_with("db error\n  caused by: ERROR: duplicate key value"));
    assert!(text.contains("DETAIL: Key (sku)=(CHAIR-01) already exists."));

    // From the client: the conversion error is the cause.
    let row = client
        .query_one("SELECT price_cents FROM products", &[])
        .await?;
    let wrong = row.try_get::<_, i32>(0).unwrap_err();
    assert_eq!(
        report(&wrong),
        "error deserializing column 0\n  caused by: cannot convert between \
         the Rust type `i32` and the Postgres type `int8`"
    );

    // Some errors have no cause at all: the category is the whole story.
    let none = client
        .query_one("SELECT 1 WHERE false", &[])
```

```
        .await
        .unwrap_err();
    assert_eq!(report(&none), "query returned an unexpected number of rows");
    assert!(none.as_db_error().is_none() && none.code().is_none());
    Ok(())
}
```

Logging `{error:?}` (`Debug`) shows the kind and the cause too, in a more technical form: `Error { kind: FromSql(0), cause: Some(WrongType { postgres: Int8, rust: "i32" }) }`. Either is fine for a log; `{error}` alone is not.

> **Pitfall:** there is no public way to ask "was this the row-count error?" or "was this a conversion error?". Only the server's errors have a code. Design so that you do not need to ask: use `query_opt` and handle `None` instead of catching `query_one`'s failure, and treat a conversion error as the bug it is.

6.2 `DbError`: the server's report

When the server reports an error, it sends far more than a message, and `DbError` keeps all of it. The fields that matter most:

Method	Content
`code()`	The SQLSTATE: five characters, the first two being the class (§6.3)
`message()`	The primary message, one line, meant for a person
`detail()`	More detail, often with the offending values: `Key (sku)=(CHAIR-01) already exists.`
`hint()`	A suggestion for fixing it, when the server has one
`severity()` / `parsed_severity()`	`ERROR` or `FATAL` (the session is over) or `PANIC`, as text or as a `Severity`
`position()`	For an error in the statement's text: where, as a 1-based character position
`schema()`, `table()`, `column()`, `datatype()`, `constraint()`	What the error is about, by name
`where_()`	The call stack, for errors raised inside functions and triggers
`routine()`, `file()`, `line()`	Where in PostgreSQL's own source the error was raised

```
use tokio_postgres::error::{ErrorPosition, Severity, SqlState};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 sku         text   PRIMARY KEY,
```

```
                name        text   NOT NULL,
                price_cents bigint CONSTRAINT price_positive CHECK (price_cents > 0)
            )",
        )
        .await?;

    // A CHECK constraint: which one, on which table, and the row that broke it.
    let error = client
        .execute("INSERT INTO products VALUES ('LAMP-01', 'Lamp', 0)", &[])
        .await
        .unwrap_err();
    let db = error.as_db_error().unwrap();
    assert_eq!(db.code(), &SqlState::CHECK_VIOLATION);
    assert_eq!(db.constraint(), Some("price_positive"));
    assert_eq!(db.table(), Some("products"));
    assert_eq!(
        db.detail(),
        Some("Failing row contains (LAMP-01, Lamp, 0).")
    );
    assert_eq!(db.parsed_severity(), Some(Severity::Error));

    // NOT NULL names the column instead of a constraint.
    let error = client
        .execute("INSERT INTO products VALUES ('LAMP-01', NULL, 1)", &[])
        .await
        .unwrap_err();
    let db = error.as_db_error().unwrap();
    assert_eq!(db.code(), &SqlState::NOT_NULL_VIOLATION);
    assert_eq!(db.column(), Some("name"));

    // A syntax error points at the place in the statement.
    let error = client.execute("SELEC 1", &[]).await.unwrap_err();
    let db = error.as_db_error().unwrap();
    assert_eq!(db.code(), &SqlState::SYNTAX_ERROR);
    assert!(matches!(db.position(), Some(ErrorPosition::Original(1))));

    // Your own errors, raised in SQL, with your own code, detail and hint.
    let error = client
        .batch_execute(
            "DO $$ BEGIN
                 RAISE EXCEPTION 'order % is already collected', 42
                     USING ERRCODE = 'SR001', DETAIL = 'collected at 12:00',
                           HINT = 'only open orders can be cancelled';
             END $$",
        )
        .await
        .unwrap_err();
    let db = error.as_db_error().unwrap();
    assert_eq!(db.code().code(), "SR001");
    assert_eq!(db.message(), "order 42 is already collected");
    assert_eq!(db.hint(), Some("only open orders can be cancelled"));
    assert!(db.where_().unwrap().contains("at RAISE"));
    Ok(())
}
```

That last example matters more than it looks. A trigger or a function can enforce a rule the schema cannot express, and report it with a code of your own. The SQL standard reserves classes starting with `0` to `4` and `A` to `H`; the rest are for implementations, and PostgreSQL itself uses several of them (`53`, `55`, `57`, `P0`, `XX`). A class it does not use, such as `SR` above, is yours, and as easy for your Rust code to match as a built-in code.

> **Pitfall:** `detail()` often contains the data that caused the error: the duplicate email address, the failing row. Returning it to an API client can leak another customer's data, and logging it can put personal data in your logs. Decide deliberately where each field goes; §6.5 shows a policy.

6.3 `SqlState`: the codes worth matching

A SQLSTATE is five characters. The first two are a *class* that groups related errors; the whole code names one condition. tokio-postgres has a constant for each code PostgreSQL defines, and the constants work directly as patterns in a `match`. Most applications need a dozen:

Class	Code	`SqlState` constant	Usually means
`23` integrity	`23505`	`UNIQUE_VIOLATION`	duplicate: 409 Conflict
	`23503`	`FOREIGN_KEY_VIOLATION`	a reference to something that does not exist (or is still referenced)
	`23502`	`NOT_NULL_VIOLATION`	a missing required value
	`23514`	`CHECK_VIOLATION`	a rule of the data: 422
	`23P01`	`EXCLUSION_VIOLATION`	an overlapping booking (exclusion constraints)
`40` transaction rollback	`40001`	`T_R_SERIALIZATION_FAILURE`	a concurrent transaction won: **retry** (Chapter 8)
	`40P01`	`T_R_DEADLOCK_DETECTED`	a deadlock, and you were chosen to fail: **retry**
`55` object state	`55P03`	`LOCK_NOT_AVAILABLE`	`lock_timeout` or `NOWAIT`: retry later
`57` operator intervention	`57014`	`QUERY_CANCELED`	`statement_timeout` or a cancel request (Chapter 12)
	`57P01`	`ADMIN_SHUTDOWN`	the server is shutting down, or your session was terminated
`53` resources	`53300`	`TOO_MANY_CONNECTIONS`	`max_connections` reached
`25` transaction state	`25P02`	`IN_FAILED_SQL_TRANSACTION`	an earlier error aborted this transaction (§6.6)
`42` syntax and access	`42P01`, `42703`	`UNDEFINED_TABLE`, `UNDEFINED_COLUMN`	a bug, or a schema the code does not match

Matching by code is precise; matching by class handles groups. A small classifier shows both:

```
use tokio_postgres::Error;
use tokio_postgres::error::SqlState;

#[derive(Debug, PartialEq)]
enum Verdict {
    Duplicate,
    BadInput,
    RetryNow,
```

```
    RetryLater,
    TooSlow,
    Bug,
    Unavailable,
}

fn classify(error: &Error) -> Verdict {
    let Some(code) = error.code() else {
        return if error.is_closed() {
            Verdict::Unavailable
        } else {
            Verdict::Bug // client-side: conversions, row counts, parameters
        };
    };
    match *code {
        SqlState::UNIQUE_VIOLATION => Verdict::Duplicate,
        SqlState::T_R_SERIALIZATION_FAILURE | SqlState::T_R_DEADLOCK_DETECTED =>
Verdict::RetryNow,
        SqlState::LOCK_NOT_AVAILABLE => Verdict::RetryLater,
        SqlState::QUERY_CANCELED => Verdict::TooSlow,
        // The rest of class 23 (integrity) and class 22 (data exceptions, like a bad
number).
        _ if code.code().starts_with("23") || code.code().starts_with("22") =>
Verdict::BadInput,
        // Class 08 (connection), 53 (resources), 57P0x (shutting down).
        _ if code.code().starts_with("08") || code.code().starts_with("53") =>
Verdict::Unavailable,
        SqlState::ADMIN_SHUTDOWN | SqlState::CANNOT_CONNECT_NOW => Verdict::Unavailable,
        _ => Verdict::Bug,
    }
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE t (id int PRIMARY KEY, n int CHECK (n > 0));
             INSERT INTO t VALUES (1, 1);",
        )
        .await?;
    let verdict = |sql: &'static str| {
        let client = &client;
        async move { classify(&client.execute(sql, &[]).await.unwrap_err()) }
    };
    assert_eq!(
        verdict("INSERT INTO t VALUES (1, 1)").await,
        Verdict::Duplicate
    );
    assert_eq!(
        verdict("INSERT INTO t VALUES (2, 0)").await,
        Verdict::BadInput
    );
    assert_eq!(verdict("SELECT 1 / 0").await, Verdict::BadInput); // 22012
    assert_eq!(verdict("SELECT * FROM nowhere").await, Verdict::Bug); // 42P01
    assert_eq!(
        verdict("SET statement_timeout = 10; SELECT pg_sleep(1)").await,
        Verdict::Bug,
        "two statements in one prepared statement: 42601"
    );
    Ok(())
}
```

Notice the last case: it looks like a timeout test, but `execute` uses the extended protocol, which takes one statement, so the server refused the text before anything ran. Even a classifier needs tests.

> **Rust refresher:** `match *code { SqlState::UNIQUE_VIOLATION => … }` matches a value against a *constant pattern*. It works because `SqlState`'s constants are plain values of a type that derives `PartialEq` and `Eq`, so the compiler can compare structurally. The `*` dereferences the `&SqlState` that `code()` returns; you could also write `Some(&SqlState::…)` against `error.code()` directly.

6.4 `is_closed`, and the error you cannot classify

`is_closed()` is true when the connection is gone: the server restarted, a network device dropped the TCP connection, an administrator terminated the session. Nothing on this client will work again, and the remedy is a new connection, which a pool does for you (Chapter 13).

There is one situation no classification can resolve. If the connection breaks while a `COMMIT` is in flight, the client cannot know whether the server committed before the connection broke. The data may or may not be there. Retrying blindly can apply a payment twice; not retrying can lose it.

> **Pitfall:** "commit outcome unknown" is rare and real. Operations that must not happen twice need a way to find out what happened: an idempotency key stored in the same transaction (`INSERT … ON CONFLICT DO NOTHING` on the key tells you on retry whether the first attempt committed), or a status the client can query. stockroom's outbox (Chapter 16) is built on the same idea: the job row commits together with the change, or not at all.

6.5 An error type for an application

A program should classify database errors once, at the boundary, into its own error type, and decide there what is shown to whom. Three rules make that work:

1. **One `From<tokio_postgres::Error>`** that looks at the code and turns the cases the application handles into variants of its own: not found, conflict, invalid, busy. Everything else stays a database error.
2. **Constraint names are part of the schema's contract.** Name every constraint a user can trip over (`CONSTRAINT price_positive CHECK (…)`), and map the name to a message. The server's text is for developers; your message is for users.
3. **The server's text goes to the log, never to the user.** It can contain table names, values and other internals (§6.2).

```
use tokio_postgres::error::SqlState;
use tokio_postgres::{Client, Error};

#[derive(Debug, thiserror::Error)]
enum AppError {
    #[error("{0} not found")]
    NotFound(&'static str),
    #[error("{0}")]
    Conflict(&'static str),
    #[error("{0}")]
    Invalid(&'static str),
```

```
    #[error("the database is busy, try again")]
    Busy(#[source] Error),
    #[error("database error")]
    Db(#[source] Error),
}

/// What each named constraint means to a user of the API.
fn constraint_message(name: &str) -> Option<&'static str> {
    Some(match name {
        "products_pkey" => "a product with this SKU already exists",
        "price_positive" => "the price must be positive",
        _ => return None,
    })
}

impl From<Error> for AppError {
    fn from(error: Error) -> Self {
        let Some(db) = error.as_db_error() else {
            return AppError::Db(error);
        };
        let message = db.constraint().and_then(constraint_message);
        match (db.code(), message) {
            (&SqlState::UNIQUE_VIOLATION, Some(message)) => AppError::Conflict(message),
            (&SqlState::CHECK_VIOLATION, Some(message)) => AppError::Invalid(message),
            (&SqlState::T_R_SERIALIZATION_FAILURE | &SqlState::T_R_DEADLOCK_DETECTED, _) =>
{
                AppError::Busy(error)
            }
            _ => AppError::Db(error),
        }
    }
}

async fn create_product(client: &Client, sku: &str, price: i64) -> Result<(), AppError> {
    client
        .execute("INSERT INTO products VALUES ($1, $2)", &[&sku, &price])
        .await?; // classified here, by the From impl
    Ok(())
}

async fn price_of(client: &Client, sku: &str) -> Result<i64, AppError> {
    let row = client
        .query_opt("SELECT price_cents FROM products WHERE sku = $1", &[&sku])
        .await?;
    Ok(row.ok_or(AppError::NotFound("product"))?.get(0))
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 sku text PRIMARY KEY,
                 price_cents bigint CONSTRAINT price_positive CHECK (price_cents > 0))",
        )
        .await?;

    create_product(&client, "CHAIR-01", 4999).await.unwrap();
    let duplicate = create_product(&client, "CHAIR-01", 1).await.unwrap_err();
    assert_eq!(
        duplicate.to_string(),
        "a product with this SKU already exists"
    );
    let free = create_product(&client, "LAMP-01", 0).await.unwrap_err();
    assert_eq!(free.to_string(), "the price must be positive");
    assert!(matches!(
        price_of(&client, "VASE-01").await,
        Err(AppError::NotFound("product"))
    ));
```

```
    // Anything unplanned keeps the original error as its source, for the log.
    let other = AppError::from(
        client
            .execute("INSERT INTO nowhere VALUES (1)", &[])
            .await
            .unwrap_err(),
    );
    assert_eq!(other.to_string(), "database error");
    let cause = std::error::Error::source(&other).unwrap();
    assert!(
        cause
            .source()
            .unwrap()
            .to_string()
            .contains("\"nowhere\" does not exist")
    );
    Ok(())
}
```

The `?` in `create_product` and `price_of` is where classification happens, through `From`. The rest of the program sees `AppErrors` with messages fit for users, and the original `tokio_postgres::Error` rides along as the `source()` of the variants that keep it, ready for the log. Chapter 15 turns each variant into an HTTP status.

6.6 An error inside a transaction

After any error, a PostgreSQL transaction refuses to do anything else until it is rolled back:

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY);
             INSERT INTO products VALUES ('CHAIR-01');",
        )
        .await?;

    let tx = client.transaction().await?;
    // One statement fails...
    let duplicate = tx
        .execute("INSERT INTO products VALUES ('CHAIR-01')", &[])
        .await;
    assert!(duplicate.is_err());
    // ...and every statement after it is refused, however harmless.
    let refused = tx.query_one("SELECT 1", &[]).await.unwrap_err();
    assert_eq!(refused.code(), Some(&SqlState::IN_FAILED_SQL_TRANSACTION)); // 25P02
    assert_eq!(
        refused.as_db_error().unwrap().message(),
        "current transaction is aborted, commands ignored until end of transaction block"
    );
    tx.rollback().await?;

    // A savepoint limits the damage: roll back to it, and the transaction carries on.
    let mut tx = client.transaction().await?;
    let attempt = tx.savepoint("attempt").await?;
    let duplicate = attempt
        .execute("INSERT INTO products VALUES ('CHAIR-01')", &[])
        .await;
    assert!(duplicate.is_err());
    attempt.rollback().await?; // ROLLBACK TO SAVEPOINT attempt
```

```
    tx.execute("INSERT INTO products VALUES ('LAMP-01')", &[])
        .await?;
    tx.commit().await?;

    let count: i64 = client
        .query_one("SELECT count(*) FROM products", &[])
        .await?
        .get(0);
    assert_eq!(count, 2);
    Ok(())
}
```

So "try the insert, and if it is a duplicate, do something else" does not work inside a transaction, unless the attempt runs in a savepoint. Chapter 8 covers savepoints properly, and stockroom's batch endpoint uses one per item. Often there is a better SQL answer that avoids the error altogether, such as `INSERT … ON CONFLICT DO NOTHING`.

> **Pitfall:** a 25P02 in your logs is never the real problem; it is the *second* error. Look for the first one, earlier in the same transaction. Code that swallows an error inside a transaction (a `let _ =` on a failed `execute`) produces 25P02s far away from the cause.

6.7 Notices are not errors

The server can also send *notices*: messages that do not stop anything. `DROP TABLE IF EXISTS` on a missing table sends one, as does `CREATE TABLE IF NOT EXISTS` on an existing table, or a `RAISE NOTICE` in a function. The statement succeeds:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    // Both succeed; each also makes the server send a notice.
    client.batch_execute("DROP TABLE IF EXISTS nowhere").await?;
    client
        .batch_execute("DO $$ BEGIN RAISE NOTICE 'stock is low'; END $$")
        .await?;
    Ok(())
}
```

Where did the notices go? A notice is not part of any query's response: it arrives on the connection like a notification would, and only the `Connection` sees it. When the connection is simply awaited, as the playground's `connect` does, tokio-postgres passes each notice to the `log` crate at the `info` level, so you see it only if your program installed a logger for `log`. To handle notices yourself, drive the connection with `poll_message` instead of awaiting it (Chapter 11). stockroom's migration runner does exactly that, which is how "relation … already exists, skipping" ends up in its log.

Side by side – handling a unique violation

The task: insert a product whose SKU already exists, and recognize the error as a duplicate, with the constraint's name.

```
-- psql
tpbook=# INSERT INTO products VALUES ('CHAIR-01', 1);
ERROR:  duplicate key value violates unique constraint "products_pkey"
DETAIL:  Key (sku)=(CHAIR-01) already exists.
tpbook=# \errverbose
ERROR:  23505: duplicate key value violates unique constraint "products_pkey"
DETAIL:  Key (sku)=(CHAIR-01) already exists.
SCHEMA NAME:  pg_temp_12
TABLE NAME:  products
CONSTRAINT NAME:  products_pkey
LOCATION:  _bt_check_unique, nbtinsert.c:673
```

```
// sqlx (illustration)
match sqlx::query("INSERT INTO products VALUES ($1,
$2)").bind("CHAIR-01").bind(1_i64).execute(&pool).await {
    Err(sqlx::Error::Database(e)) if e.is_unique_violation() => {
        assert_eq!(e.constraint(), Some("products_pkey"));
    }
    other => { /* … */ }
}
```

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999);",
        )
        .await?;
    match client
        .execute("INSERT INTO products VALUES ('CHAIR-01', 1)", &[])
        .await
    {
        Err(e) if e.code() == Some(&SqlState::UNIQUE_VIOLATION) => {
            let constraint = e.as_db_error().and_then(|db| db.constraint());
            assert_eq!(constraint, Some("products_pkey"));
        }
        other => panic!("expected a duplicate, got {other:?}"),
    }
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	The SQLSTATE, the constraint, the detail: everything the server sent	The same fields, from the same message	The same fields, from the same message
Different	Shows the message and detail; `\errverbose` shows every field of the last error	An `Error::Database` variant, with helpers such as `is_unique_violation()`; other errors are separate variants of one public enum	One opaque `Error`: `code()` and `as_db_error()` for server errors, the cause in `source()` for the rest, and `Display` that is only a category

In stockroom

stockroom classifies database errors in exactly one place, error.rs, with the shape of §6.5 and a few more variants:

```
// stockroom/src/error.rs (excerpt)
impl From<tokio_postgres::Error> for AppError {
    fn from(error: tokio_postgres::Error) -> Self {
        let Some(db) = error.as_db_error() else {
            // Not from the server: IO, a closed connection, or a conversion on our side.
            return if error.is_closed() {
                AppError::Unavailable("the connection to the database was lost".into())
            } else {
                AppError::Db(error)
            };
        };
        match *db.code() {
            SqlState::UNIQUE_VIOLATION => {
                let (field, message) =
                    constraint_message(db).unwrap_or((None, "this already exists"));
                AppError::Conflict {
                    field,
                    message: message.into(),
                }
            }
            // … CHECK, NOT NULL and FOREIGN KEY violations become AppError::Invalid …
            SqlState::T_R_SERIALIZATION_FAILURE
            | SqlState::T_R_DEADLOCK_DETECTED
            | SqlState::LOCK_NOT_AVAILABLE => AppError::Busy(error),
            SqlState::QUERY_CANCELED => AppError::Timeout(error),
            SqlState::ADMIN_SHUTDOWN
            | SqlState::CANNOT_CONNECT_NOW
            | SqlState::TOO_MANY_CONNECTIONS =>
AppError::Unavailable(db.message().to_owned()),
            _ => AppError::Db(error),
        }
    }
}
```

`constraint_message` maps every named constraint in the schema to the request field it is about and a message, so a client gets `{"error": "conflict", "field": "sku", "message": "a product with this SKU already exists"}` instead of a server message. `Busy` is what the transaction helper of Chapter 8 retries. When an error *is* the server's fault (a 5xx), its fields go to the log as structured data and the client gets a generic message:

```
// stockroom/src/error.rs (excerpt)
fn log_server_error(error: &AppError) {
    let db = match error {
        AppError::Busy(e) | AppError::Timeout(e) | AppError::Db(e) => e.as_db_error(),
        _ => None,
    };
    match db {
        Some(db) => tracing::error!(
            code = db.code().code(),
            message = db.message(),
            detail = db.detail(),
            hint = db.hint(),
            table = db.table(),
            constraint = db.constraint(),
            "{error}"
        ),
        None => tracing::error!(error = ?error, "{error}"),
    }
}
```

There is one deliberate exception to "server text never reaches the client": errors in an uploaded CSV file (class 22, data exceptions, raised by `COPY`). Their message and position quote the client's own data back, "invalid input syntax for type bigint: "cheap" (COPY product_import, line 3, column price_cents: "cheap")", which is exactly what the uploader needs. repo/products.rs makes that exception in a function of its own, `input_error`, so that it cannot spread by accident.

Summary

- Every failure is a `tokio_postgres::Error`. Its `Display` is a category (`db error`); the explanation is in `source()`. Log `{:?}`, or walk the source chain.
- `as_db_error()` and `code()` exist for errors the server reported; client-side errors (row counts, conversions, parameters) have no code and no public kind.
- `DbError` carries the SQLSTATE, message, detail, hint, position, the schema, table, column, datatype and constraint involved, and the call stack of functions and triggers.
- Match `SqlState` constants as patterns; match classes (`23`, `40`, `08`) by prefix.
- `is_closed()` means a new connection is needed. A broken connection during `COMMIT` leaves the outcome unknown: design operations to be safely retried.
- Classify once, in a `From<tokio_postgres::Error>` for your own error type. Name your constraints, and map the names to messages. Keep the server's text out of responses.
- After an error, a transaction is aborted (25P02) until rolled back; savepoints contain the damage.
- Notices are not errors. When the connection is just awaited they go to the `log` crate; `poll_message` hands them to you.

Exercises

1. **A better log line.** Write `fn describe(error: &tokio_postgres::Error) -> String` that produces `23505 unique_violation on products (products_pkey): duplicate key value …` for server errors and falls back to the source chain for the others. *Hint:* the SQLSTATE's name is not in tokio-postgres; a short `match` on the codes you care about is enough.
2. **Retry or not?** For each, say whether a request that failed with it should be retried as is, retried after a delay, or not retried: `40001`, `40P01`, `55P03`, `57014`, `23505`, `53300`, `57P01`, a closed connection during `SELECT`, a closed connection during `COMMIT`.
3. **A rule in a trigger.** Write a trigger that refuses to delete a product that has stock, with `RAISE EXCEPTION … USING ERRCODE = 'SR002'`, and map `SR002` to a 409 Conflict variant of your `AppError`.
4. **Find the first error.** A colleague's code logs "current transaction is aborted" every few minutes. What do you look for, and what is a likely cause in Rust code?

Solutions

2. `40001` and `40P01`: retry the whole transaction, immediately or after a short random pause (they are caused by concurrency, not by the request). `55P03` (lock timeout): retry after a delay, or report "busy". `57014`: do not retry blindly: it was too slow once, and will probably be too slow again; report

it, and look at the query. `23505`: never; the request conflicts with data. `53300` and `57P01`: retry after a delay, preferably on another connection or server. A closed connection during `SELECT`: retry on a new connection, since reading is safe to repeat. A closed connection during `COMMIT`: the outcome is unknown; only retry if the operation is idempotent or you can check what happened first (§6.4).

4. Look for the error *before* it, in the same transaction, on the same connection: the 25P02 is the consequence. In Rust code, the classic cause is an error that was caught and ignored inside a transaction, such as `let _ = tx.execute(…).await;` or a `match` that logs and continues, after which the transaction is used again. Either use a savepoint around the statement that may fail, or let the error end the transaction.